

UNIT V

Testing: Strategies, A Strategic Approach to Conventional Software Testing, Test Strategies for O-O Software.

Tactics: Software Testing Fundamentals, Black-box and White-box Testing, Basis Path Testing, Control Structure Testing, O-O Testing Methods.

Debugging: Debugging Techniques, The Art of Debugging.

Product Metrics: A Framework for Product Metrics, Metrics for each phase of software development.

Software Quality: Definition, **Quality Assurance:** Basic Elements, Tasks, Formal Approaches, Statistical Software Quality Assurance, Software Reliability, ISO 9000 Quality Standards, SQA Plan.

TESTING STRATEGIES

Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user. Testing shows errors, requirements conformance, performance & indication of quality.

A STRATEGIC APPROACH TO SOFTWARE TESTING

Test Strategy incorporates test planning, test case design, test execution and resultant data collection and execution.

All provide the software developer with a template for testing and all have the following **generic characteristics**:

- Perform Formal Technical reviews(FTR) to uncover errors during software development
- Begin testing at component level and move outward to integration of entire component based system.
- Adopt testing techniques relevant to stages of testing
- Testing can be done by software developer and independent testing group
- Testing and debugging are different activities. Debugging follows testing

1.Verification & Validation

Verification refers to the set of activities that ensure that software correctly implements a specific function.

Validation refers to a different set of activities that ensure that the software that has been built is traceable to customer requirements.

Boehm [BOE81] states this way:

Verification: "Are we building the product right?"

Validation: "Are we building the right product?"

The definition of V&V encompasses many of the activities that we have referred to as software quality assurance (SQA). Verification and validation encompasses a wide array of SQA activities that include formal technical reviews, quality and configuration audits, performance monitoring, simulation,

feasibility study, documentation review, database review, algorithm analysis, development testing, qualification testing, and installation testing. Although testing plays an extremely important role in V&V, many other activities are also necessary.

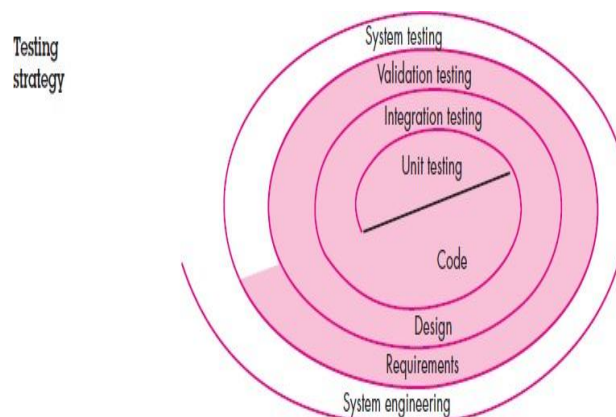
2. Organizing for Software Testing: The people who have built the software are now asked to test the software. This seems harmless in itself; after all, who knows the program better than its developers do? Unfortunately, these same developers have a vested interest in demonstrating that the program is error free, that it works according to customer requirements.

From a psychological point of view, software analysis and design (along with coding) are constructive tasks. From the point of view of the builder, testing can be considered to be (psychologically) destructive. The software developer is always responsible for testing the individual units (components) of the program, ensuring that each performs the function for which it was designed. In many cases, the developer also conducts integration testing—a testing step that leads to the construction (and test) of the complete program structure. Only after the software architecture is complete does an independent test group get involved.

3. Software Testing Strategy—The Big Picture:

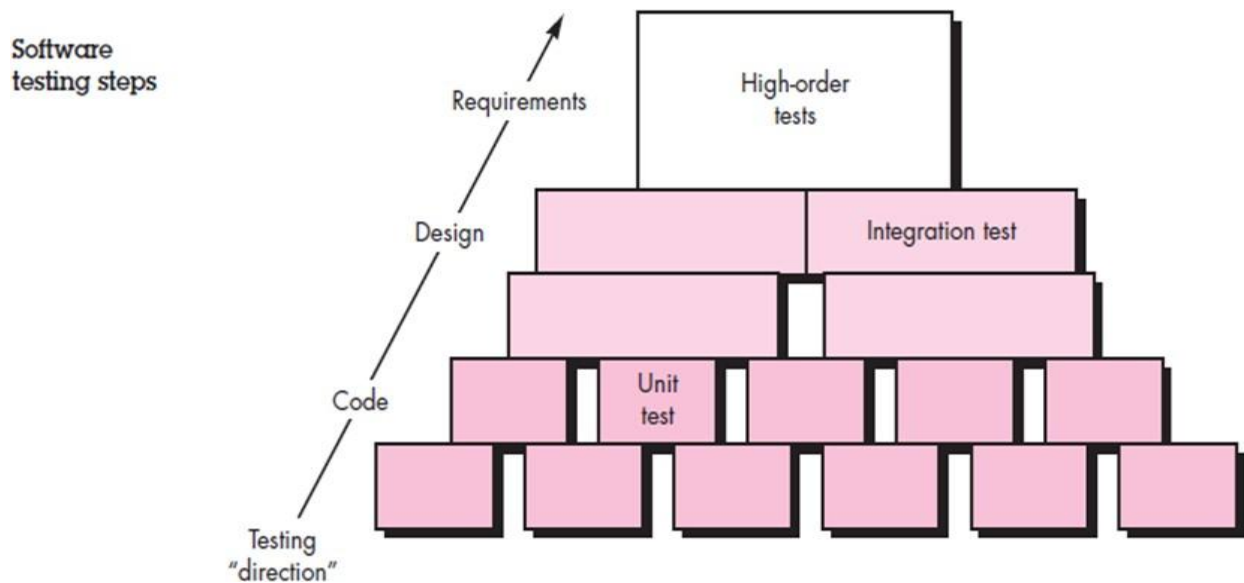
The software engineering process may be viewed as the spiral illustrated in below Figure. Initially, system engineering defines the role of software and leads to software requirements analysis, where the information domain, function, behavior, performance, constraints, and validation criteria for software are established.

Unit testing begins at the vortex of the spiral and concentrates on each unit (i.e., component) of the software as implemented in source code. Testing progresses by moving outward along the spiral to **integration testing**, where the focus is on design and the construction of the software architecture. Taking another turn outward on the spiral, we encounter **validation testing**, where requirements established as part of software requirements analysis are validated against the software that has been constructed. Finally, we arrive at **system testing**, where the software and other system elements are tested as a whole.



Unit testing makes heavy use of white-box testing techniques. Black-box test case design techniques are the most prevalent **during integration**, although a limited amount of white-box testing may be used to ensure coverage of major control paths. Black-box testing techniques are used exclusively during **validation**. Software, once validated, must be combined with other system elements (e.g.,

hardware, people, and databases). **System testing** verifies that all elements mesh properly and that overall system function/performance is achieved.

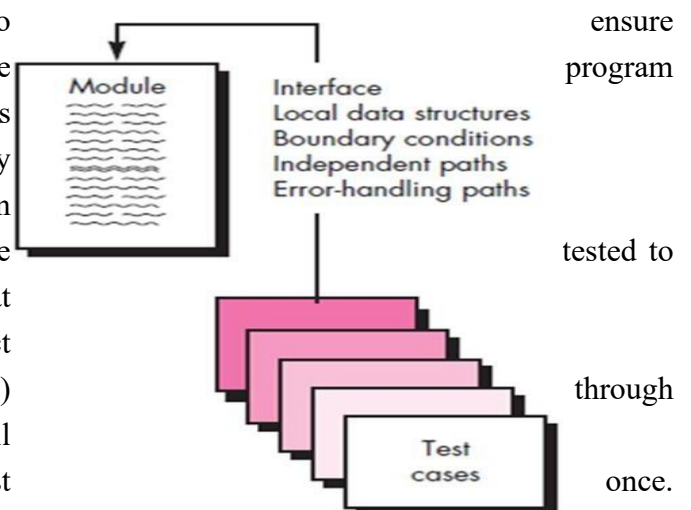


A STRATEGIC APPROACH TO CONVENTIONAL SOFTWARE TESTING

1. Unit Testing:

Unit testing focuses verification effort on the smallest unit of software design — the software component or module. Using the component-level design description as a guide, important control paths are tested to uncover errors within the boundary of the module. The unit test is white-box oriented.

i)Unit Test Considerations: The tests that occur as part of unit tests are illustrated schematically in the below figure. The module interface is tested to ensure program that information properly flows into and out of the unit under test. The local data structure is examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithm's execution. Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restrict processing. All independent paths (basis paths) through the control structure are exercised to ensure that all statements in a module have been executed at least once. And finally, all error handling paths are tested.



What errors are commonly found during Unit Testing?

(1) Misunderstood or incorrect arithmetic precedence, (2) mixed mode operations, (3) incorrect initialization, (4) precision inaccuracy, (5) incorrect symbolic representation of an expression. Comparison and control flow are closely coupled to one another (i.e., change of flow frequently occurs

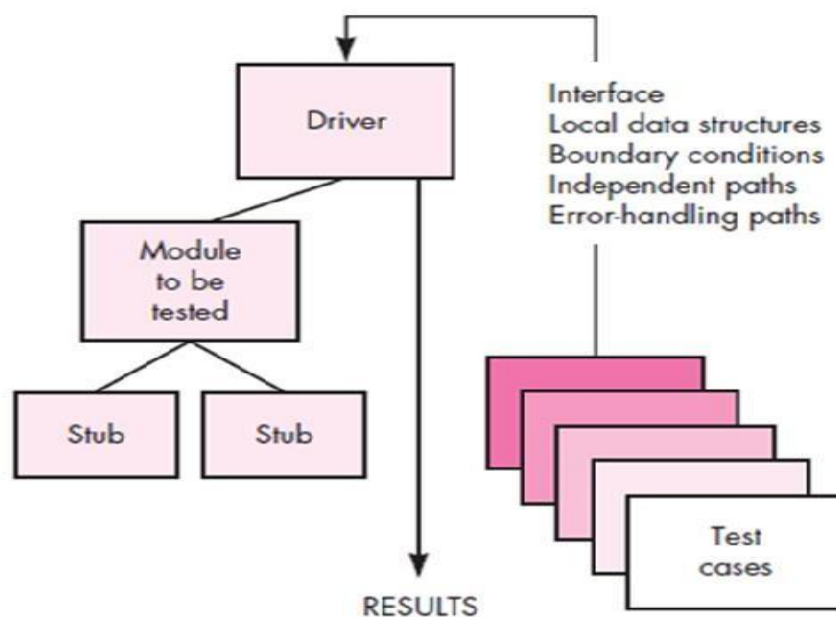
after a comparison).

Test cases should uncover errors such as

(1) comparison of different data types, (2) incorrect logical operators or precedence, (3) expectation of equality when precision error makes equality unlikely, (4) incorrect comparison of variables, (5) improper or nonexistent loop termination, (6) failure to exit when divergent iteration is encountered, and (7) improperly modified loop variables.

ii) Unit Test Procedures: After source level code has been developed, reviewed, and verified for correspondence to component-level design, unit test case design begins.

Because a component is not a stand-alone program, driver and/or stub software must be developed for each unit test. The unit test environment is illustrated in Figure 18.5. In most applications a driver is nothing more than a "main program" that accepts test case data, passes such data to the component (to be tested), and prints relevant results. Stubs serve to replace modules that are subordinate (called by) the component to be tested. A stub or "dummy subprogram" uses the subordinate module's interface, may do minimal data manipulation, prints verification of entry, and returns control to the module undergoing testing. ***Unit testing is simplified when a component with high cohesion is designed.***



2. Integration Testing:

The objective is to take unit tested components and build a program structure that has been dictated by design.

Approaches:

- ✓ **"Big bang" approach:** All components are combined in advance. Entire program is tested as a whole.
- ✓ **Incremental integration:** The program is constructed and tested in small increments, where errors are easier to isolate and corrected.

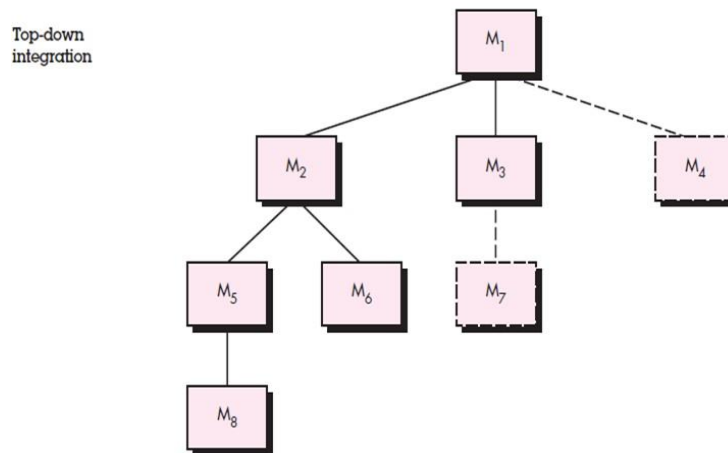
Number of different incremental integration strategies are discussed:

i) Top-Down Integration

Modules are integrated by moving downward through the control hierarchy, beginning with the main control module (main program). Modules subordinate to the main control module are incorporated into the structure **in either a depth-first or a breadth-first manner**.

Referring to figure ***Depth-first integration*** would integrate all components on a major control path of the structure. Selection of a major path and depends on application-specific characteristics. For example, selecting left hand path, components M1, M2, M5 would be integrated first. Next, M8 or (if necessary for proper functioning of M2) M6 would be integrated. Then, central and right-hand control paths are built.

Breadth-first integration incorporates all components directly subordinate at each level, moving across the structure horizontally. From the figure, components M2, M3, and M4 (a replacement for stub S4) would be integrated first. The next control level, M5, M6, and so on, follows.



Steps for Top-Down Integration:

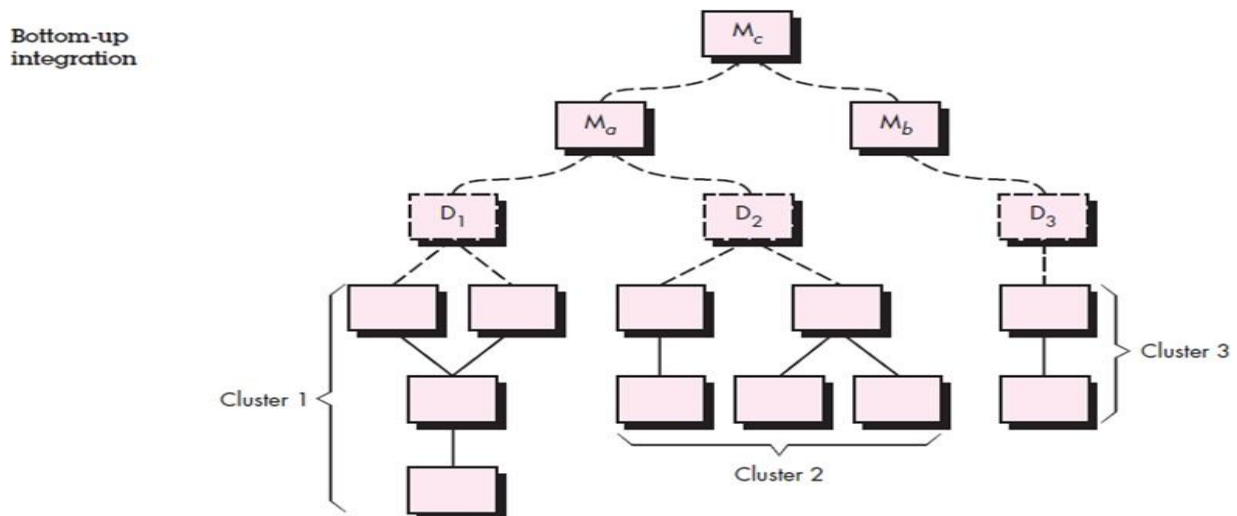
1. The main control module is used as a test driver and stubs are substituted for all components directly subordinate to the main control module.
2. Depending on the integration approach selected (i.e., depth or breadth first), subordinate stubs are replaced one at a time with actual components.
3. Tests are conducted as each component is integrated.
4. On completion of each set of tests, another stub is replaced with the real component.
5. Regression testing may be conducted to ensure that new errors have not been introduced. The process continues from step 2 until the entire program structure is built.

What problems are encountered when top-down integration strategy is chosen?

The most common of these problems occurs when processing at low levels in the hierarchy is required to adequately test upper levels. Stubs replace low-level modules at the beginning of top-down testing; therefore, no significant data can flow upward in the program structure.

ii) Bottom-Up Integration

Bottom-up integration testing, as its name implies, begins construction and testing with atomic modules (i.e., components at the lowest levels in the program structure).



Steps for Bottom-Up Integration:

1. Low-level components are combined into clusters (sometimes-called builds) that perform a specific software sub function.
2. A driver (a control program for testing) is written to coordinate test case input and output.
3. The cluster is tested.
4. Drivers are removed and clusters are combined moving upward in the program structure.

Components are combined to form clusters 1, 2, and 3. Each of the clusters is tested using a driver (shown as a dashed block). Components in clusters 1 and 2 are subordinate to M_a . Drivers D_1 and D_2 are removed and the clusters are interfaced directly to M_a . Similarly, driver D_3 for cluster 3 is removed prior to integration with module M_b . Both M_a and M_b will ultimately be integrated with component M_c , and so forth.

Regression Testing: Each time a new module is added as part of integration testing, the software changes. New data flow paths are established, new I/O may occur, and new control logic is invoked. **Regression testing** is the re-execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects.

The regression test suite (the subset of tests to be executed) contains three different classes of test cases:

- A representative sample of tests that will exercise all software functions.
- Additional tests that focus on software functions that are likely to be affected by the change.
- Tests that focus on the software components that have been changed.

Smoke Testing: Smoke testing is an integration testing approach that is commonly used when “shrink- wrapped” software products are being developed. It is designed as a pacing mechanism for time-critical projects. The smoke testing approach encompasses the following activities:

1. **Software components that have been translated into code are integrated into a “build.”** A build includes all data files, libraries, reusable modules, and engineered components that are required to

implement one or more product functions.

2. ***A series of tests is designed to expose errors that will keep the build from properly performing its function:*** The intent should be to uncover “show stopper” errors that have the highest likelihood of throwing the software project behind schedule.
3. ***The build is integrated with other builds and the entire product (in its current form) is smoke tested daily:*** The integration approach may be top down or bottom up
Smoke testing provides a number of benefits when it is applied on complex, time-critical software engineering projects:
 - Integration risk is minimized.
 - The quality of the end-product is improved.
 - Progress is easier to assess.

Strategic options: The major disadvantage of the top-down approach is the need for stubs and the attendant testing difficulties that can be associated with them. The major disadvantage of bottom-up integration is that "the program as an entity does not exist until the last module is added" Selection of an integration strategy depends upon software characteristics and, sometimes, project schedule. In general, a combined approach (sometimes-called ***sandwich testing***) that uses top-down tests for upper levels of the program structure, coupled with bottom-up tests for subordinate levels may be the best compromise.

What is a critical module and why should we identify it?

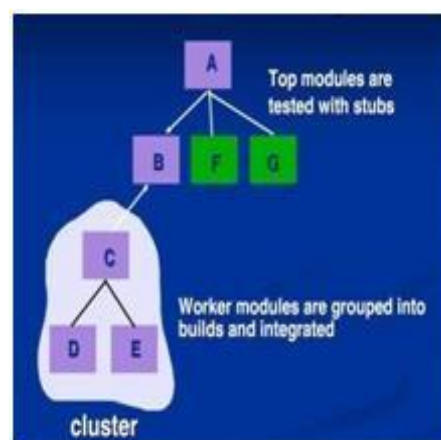
- Address several software requirements
- Has a high level of control (high in the program structure)
- Is complex or error prone
- Has definite performance requirements

Sandwich Integration

Sandwich integration is a combination of both top-down and bottom-up integration. It occurs both at the highest level modules and at the lowest level modules.

Proceeds using functional groups of modules, with each group completed before the next

- High and low-level modules are grouped based on the control and data processing they provide for a specific program feature
- Integration within the group progresses in alternating steps between the high and low level modules of the group
- When integration for a certain functional group is complete, integration and testing moves onto the next group
- Uses the advantages of both types of integration while minimizing the need for drivers and stubs



- Requires a disciplined approach so that integration doesn't tend towards the “big bang” scenario.

Fig: Sandwich Integration

Integration test work products:

An overall plan for integration of the software and a description of specific tests is documented in a Test Specification. This work product incorporates a test plan and a test procedure and becomes part of the software configuration. Testing is divided into phases and builds that address specific functional and behavioral characteristics of the software.

TEST STRATEGIES FOR OBJECT-ORIENTED SOFTWARE

The objective of testing, stated simply, is to find the greatest possible number of errors with a manageable amount of effort applied over a realistic time span.

- Although this fundamental objective remains unchanged for object oriented software.
- The nature of object-oriented software changes both testing strategy and testing tactics (Plan).

Unit Testing in the OO Context:

- When object-oriented software is considered, the concept of the unit changes. Encapsulation drives the definition of classes and objects.
- This means that each class and each instance of a class packages attributes (data) and the operations that manipulate these data.

An encapsulated class is usually the focus of unit testing.

- However, operations (methods) within the class are the smallest testable units. Because a class can contain a number of different operations, and a particular operation may exist as part of a number of different classes.
- Class testing for OO software is the equivalent of unit testing for conventional software.
- Unlike unit testing of conventional software, which tends to focus on the algorithmic detail of a module and the data that flow across the module interface,
- Class testing for OO software is driven by the operations encapsulated by the class and the State behavior of the class.

Integration Testing in the OO Context :

- Different strategies for integration testing of OO systems.
 - ✓ Thread-based testing
 - ✓ use-based testing
 - ✓ Cluster testing
- ***The first, thread-based testing***, integrates the set of classes required to respond to one input or event for the system.
 - ✓ Each thread is integrated and tested individually.
 - ✓ Regression testing is applied to ensure that no side effects occur.
- ***The second integration approach, use-based testing***, begins the construction of the system by testing those classes (called independent classes) that use very few (if any) server classes.
- After the independent classes are tested, the next layer of classes, called dependent classes, that use the independent classes are tested.

- This sequence of testing layers of dependent classes continues until the entire system is constructed.

Cluster testing:

- Cluster testing is one step in the integration testing of OO software.
- Here, a cluster of collaborating classes is exercised by designing test cases that attempt to uncover errors in the collaborations.

VALIDATION TESTING

Validation succeeds when software functions in a manner that can be reasonably expected by the customer.

Validation Test Criteria: Software validation is achieved through a series of black-box tests that demonstrate conformity with requirements. Both plan and procedure are designed to ensure that all functional requirements are satisfied, all behavioral characteristics are achieved, all performance requirements are attained, documentation is correct and other requirements are met.

After each validation test case has been conducted, one of two possible conditions exist:

1. The function or performance characteristics conform to specification and are accepted
2. A deviation from specification is uncovered and a deficiency list is created.

Configuration Review: An important element of the validation process is a configuration review. The intent of the review is to ensure that all elements of the software configuration have been properly developed, are cataloged, and have the necessary detail to bolster the support phase of the software life cycle. The configuration review, sometimes called an audit.

Alpha and Beta Testing: A customer conducts the **alpha test** at the developer's site. The **beta test** is conducted at one or more customer sites by the end-user of the software. Unlike alpha testing, the developer is generally not present.

SYSTEM TESTING

Software is incorporated with other system elements (e.g., hardware, people, information), and a series of system integration and validation tests are conducted. These tests fall outside the scope of the software process and are not conducted solely by software engineers. A classic system-testing problem is "finger-pointing." This occurs when an error is uncovered, and each system element developer blames the other for the problem.

- 1. Recovery Testing:** Recovery testing is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed.
- 2. Security Testing:** Security testing attempts to verify that protection mechanisms built into a system. During security testing, the tester plays the role(s) of the individual who desires to penetrate the system.
- 3. Stress Testing:** Stress testing executes a system in a manner that demands resources in abnormal quantity, frequency, or volume. For example, (1) special tests may be designed that generate ten interrupts per second, when one or two is the average rate, (2) input data rates may be increased by an order of magnitude to determine how input functions will respond, (3) test cases

that require maximum memory or other resources are executed, Essentially, the tester attempts to break the program.

A variation of stress testing is a technique called **sensitivity testing**. Sensitivity testing attempts to uncover data combinations within valid input classes that may cause instability or improper processing

- 4. Performance Testing:** Performance testing is designed to test the run-time performance of software within context of an integrated system. Performance testing occurs throughout all steps in testing process. Performance tests are often coupled with stress testing and usually require both hardware and software instrumentation.

SOFTWARE TESTING FUNDAMENTALS

The goal of testing is to find errors, and a good test is one that has a high probability of finding an error. Therefore, you should design and implement a computer based system or a product with “testability” in mind. At the same time, the tests themselves must exhibit a set of characteristics that achieve the goal of finding the most errors with a minimum of effort.

1. **Testability:** “Software testability is simply how easily [a computer program] can be tested.” The following characteristics lead to testable software.

Operability: “The better it works, the more efficiently it can be tested.”

Observability: “What you see is what you test.”

Controllability: “The better we can control the software, the more the testing can be automated and optimized.”

Controllability: “The better we can control the software, the more the testing can be automated and optimized.”

Controllability: “The better we can control the software, the more the testing can be automated and optimized.”

Stability: “The fewer the changes, the fewer the disruptions to testing.”

Understandability: “The more information we have, the smarter we will test.”

2. Test Characteristics:

A good test has a high probability of finding an error.

A good test is not redundant.

A good test has a high probability of finding an error.

A good test has a high probability of finding an error.

BLACK-BOX TESTING

Black-box testing, also called behavioral testing, focuses on the functional requirements of the software. Black-box testing enables the software engineer to derive sets of input conditions that will fully exercise all functional requirements for a program.

Black-box testing attempts to find errors in the following categories:

- (1) incorrect or missing functions
- (2) interface errors
- (3) errors in data structures or external data base access
- (4) behavior or performance errors
- (5) initialization and termination errors.

White-box testing is performed early in the testing process; Black-box testing tends to be applied during later stages of testing.

GRAPH-BASED TESTING METHODS: Software testing begins by creating a graph of important objects and their relationships and then devising a series of tests that will cover the graph so that each object and relationship is exercised and errors are uncovered.

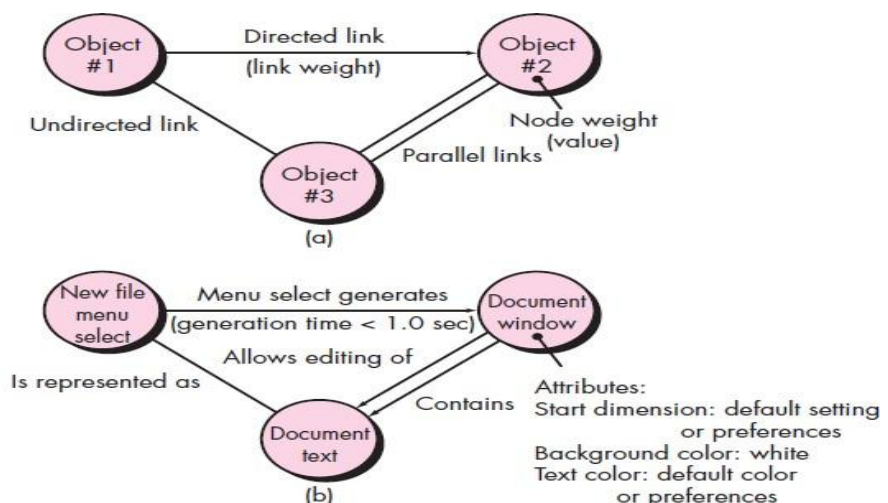
A graph is a collection of nodes that represent **objects**; links that represent the **relationships between objects**; node weights that describe the **properties of a node** (e.g., a specific data value or state behavior); and link weights that describe some **characteristic of a link**.

A **directed link** (represented by an arrow) indicates that a relationship moves in only one direction.

A **bidirectional link**, also called a symmetric link, implies that the relationship applies in both directions.

Parallel links are used when a number of different relationships are established between graph nodes.

(a) Graph notation; (b) simple example



Referring to the figure, a menu select on **new file** generates a **document window**. The node weight of **document window** provides a list of the window attributes that are to be expected when the window is generated. The link weight indicates that the window must be generated in less than 1.0 second. An undirected link establishes a symmetric relationship between the **new file menu select** and **document text**, and parallel links indicate relationships between **document window** and **document text**.

The software engineer then derives test cases by traversing the graph and covering each of the relationships shown. These test cases are designed in an attempt to find errors in any of the relationships.

EQUIVALENCE PARTITIONING: It is a black-box testing method that divides the input domain of a program into classes of data from which test cases can be derived. Test case design for equivalence partitioning is based on an evaluation of equivalence classes for an input condition.

If a set of objects can be linked by relationships that are symmetric, transitive, and reflexive, an **equivalence class** is present. An **equivalence class** represents a set of valid or invalid states for input conditions.

Typically, an input condition is a specific numeric value, a range of values, a set of related values, or a Boolean condition. Equivalence classes may be defined according to the following guidelines:

1. If an input condition **specifies a range**, one valid and two invalid equivalence classes are defined.
2. If an input condition requires a **specific value**, one valid and two invalid equivalence classes are defined.
3. If an input condition **specifies a member of a set**, one valid and one invalid equivalence class are defined.

Test cases are selected so that largest numbers of attributes of an equivalence class are exercised at once.

BOUNDARY VALUE ANALYSIS: Number of errors tends to occur at the boundaries of the input domain rather than in the "center". **Boundary value analysis** leads to a selection of test cases that exercise bounding values.

BVA leads to the selection of test cases at the "edges" of the class. Rather than focusing solely on input conditions, BVA derives test cases from the output domain as well

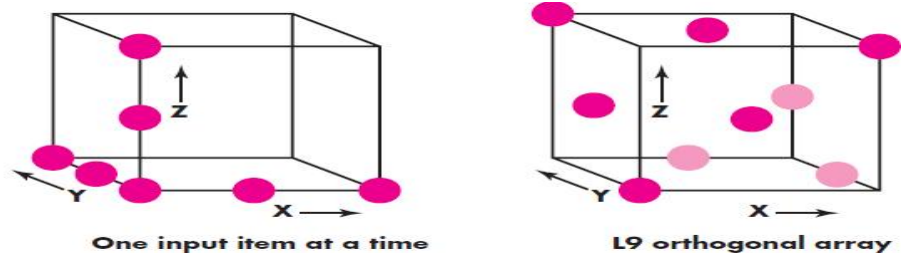
How do I create BVA test cases?

1. If an input condition specifies a range bounded by values a and b, test cases should be designed with values a and b and just above and just below a and b.
2. If an input condition specifies a number of values, test cases should be developed that exercise the minimum and maximum numbers. Values just above and below minimum and maximum are also tested.
3. Apply guidelines 1 and 2 to output conditions. For example, assume that a temperature vs. pressure table is required as output from an engineering analysis program. Test cases should be designed to create an output report that produces the maximum (and minimum) allowable number of table entries.
4. If internal program data structures have prescribed boundaries (e.g., an array has a defined limit of 100 entries), be certain to design a test case to exercise the data structure at its boundary.

ORTHOGONAL ARRAY TESTING: The orthogonal array testing method is particularly useful in finding errors associated with region faults—an error category associated with faulty logic within a software component.

To

A geometric view of test cases
Source: [Pha97]



illustrate the difference between orthogonal array testing and more conventional “one input item at a time” approaches, consider a system that has three input items, X, Y, and Z.

Each of these input items has three discrete values associated with it. There are $3^3 = 27$ possible test cases. Phadke suggests a geometric view of the possible test cases associated with X, Y, and Z illustrated in Figure. Referring to the figure, one input item at a time may be varied in sequence along each input axis. This results in relatively limited coverage of the input domain (represented by the left-hand cube in the figure).

When orthogonal array testing occurs, an L9 orthogonal array of test cases is created. The L9 orthogonal array has a balancing property that is; test cases (represented by black dots in the figure) are “dispersed uniformly throughout the test domain,”

To illustrate the use of the L9 orthogonal array, consider the send function for a fax application. Four parameters, P1, P2, P3, and P4, are passed to the send function. Each takes on three discrete values. For example, P1 takes on values:

P1 = 1, send it now

P1 = 2, send it one hour later P1 = 3, send it after midnight

P2, P3, and P4 would also take on values of 1, 2 and 3, signifying other send functions.

If a “one input item at a time” testing strategy were chosen, the following sequence of tests (P1, P2, P3, P4) would be specified: (1, 1, 1, 1), (2, 1, 1, 1), (3, 1, 1, 1), (1, 2, 1, 1), (1, 3, 1, 1), (1, 1, 2, 1), (1, 1, 3, 1), (1, 1, 1, 2), and (1, 1, 1, 3).

Phadke assesses these test cases in the following manner:

Test case	Test parameters			
	P1	P2	P3	P4
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1

Fig: An L9 Orthogonal Array

WHITE-BOX TESTING

White-box testing, sometimes called glass-box testing. Our goal is to ensure that all statements and conditions have been executed at least once. Using white-box testing methods, the software engineer can derive test cases that:

1. Guarantee that all independent paths within a module have been exercised at least once.
2. Exercise all logical decisions on their true and false sides.
3. Execute all loops at their boundaries and within their operational bounds.
4. Exercise internal data structures to ensure their validity.

White box testing techniques analyze the internal structures the used data structures, internal design, code structure, and the working of the software rather than just the functionality as in black box testing. It is also called glass box testing or clear box testing or structural testing. White Box Testing is also known as transparent testing or open box testing.

White box testing is a software testing technique that involves testing the internal structure and workings of a software application. The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.

White box testing is also known as structural testing or code-based testing, and it is used to test the software's internal logic, flow, and structure. The tester creates test cases to examine the code paths and logic flows to ensure they meet the specified requirements.

1) Process of White Box Testing:

Input: Requirements, Functional specifications, design documents, source code.

Processing: Performing risk analysis to guide through the entire process.

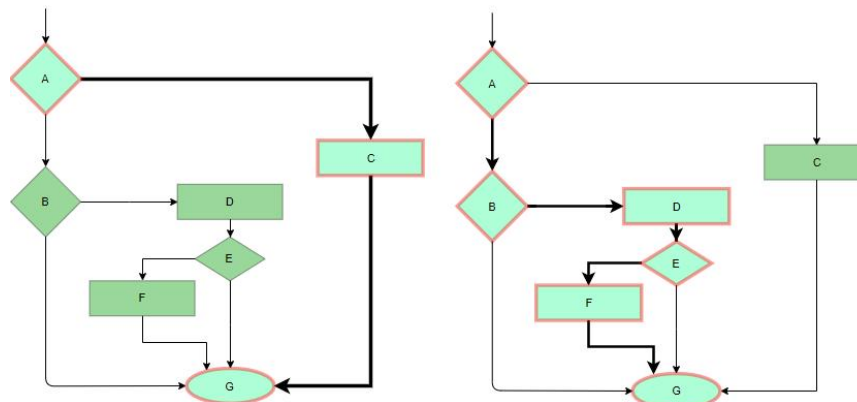
Proper test planning: Designing test cases to cover the entire code. Execute rinse-repeat until error-free software is reached. Also, the results are communicated.

Output: Preparing final report of the entire testing process.

2) Testing Techniques:

a. Statement Coverage:

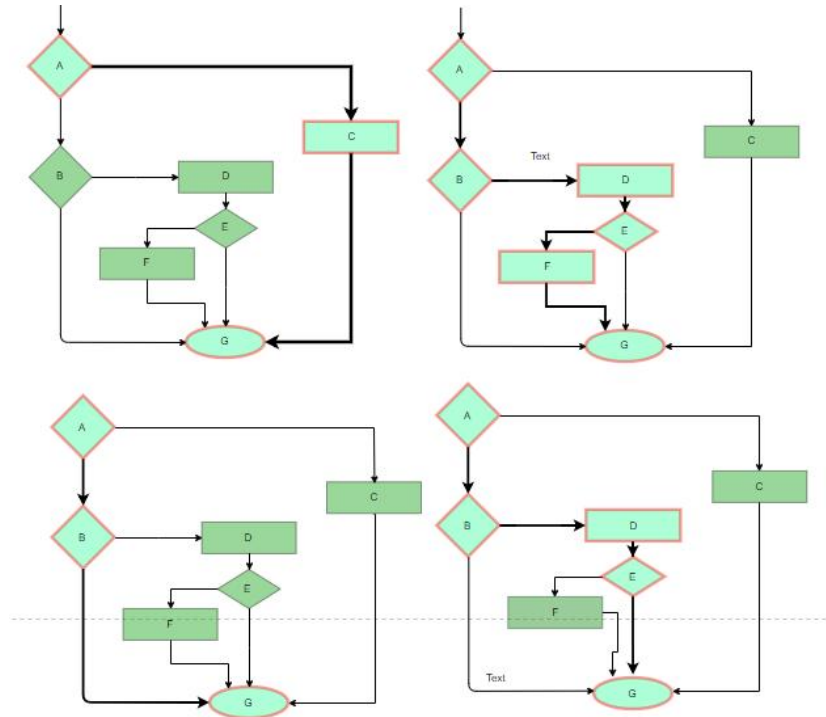
In this technique, the aim is to traverse all statements at least once. Hence, each line of code is tested. In the case of a flowchart, every node must be traversed at least once. Since all lines of code are covered, it helps in pointing out faulty code.



Statement Coverage Example

b. Branch Coverage:

In this technique, test cases are designed so that each branch from all decision points is traversed at least once. In a flowchart, all edges must be traversed at least once.



4 test cases are required such that all branches of all decisions are covered, i.e, all edges of the flowchart are covered.

c. Condition Coverage :

In this technique, all individual conditions must be covered as shown in the following example:

```
READ X, Y
IF(X == 0 || Y == 0)
PRINT '0'
#TC1 – X = 0, Y = 55
#TC2 – X = 5, Y = 0
```

d. Multiple Condition Coverage:

In this technique, all the possible combinations of the possible outcomes of conditions are tested at least once. Let's consider the following example:

```
READ X, Y
IF(X == 0 || Y == 0)
PRINT '0'
#TC1: X = 0, Y = 0
#TC2: X = 0, Y = 5
#TC3: X = 55, Y = 0
#TC4: X = 55, Y = 5
```

e. Basis Path Testing:

In this technique, control flow graphs are made from code or flowchart and then Cyclomatic complexity is calculated which defines the number of independent paths so that the minimal number of test cases can be designed for each independent path. Steps:

- ✓ Make the corresponding control flow graph
- ✓ Calculate the cyclomatic complexity
- ✓ Find the independent paths
- ✓ Design test cases corresponding to each independent path
- ✓ $V(G) = P + 1$, where P is the number of predicate nodes in the flow graph
- ✓ $V(G) = E - N + 2$, where E is the number of edges and N is the total number of nodes
- ✓ $V(G) = \text{Number of non-overlapping regions in the graph}$
- ✓ #P1: 1 – 2 – 4 – 7 – 8
- ✓ #P2: 1 – 2 – 3 – 5 – 7 – 8
- ✓ #P3: 1 – 2 – 3 – 6 – 7 – 8
- ✓ #P4: 1 – 2 – 4 – 7 – 1 – . . . – 7 – 8

f. Loop Testing:

Loops are widely used and these are fundamental to many algorithms hence, their testing is very important. Errors often occur at the beginnings and ends of loops.

Simple loops: For simple loops of size n, test cases are designed that:

1. Skip the loop entirely
2. Only one pass through the loop
3. 2 passes
4. m passes, where $m < n$
5. n-1 and n+1 passes

Nested loops: For nested loops, all the loops are set to their minimum count, and we start from the innermost loop. Simple loop tests are conducted for the innermost loop and this is worked outwards till all the loops have been tested.

Concatenated loops: Independent loops, one after another. Simple loop tests are applied for each. If they're not independent, treat them like nesting.

3) White Testing is performed in 2 Steps:

Tester should understand the code well

Tester should write some code for test cases and execute them

4) Tools required for White box testing:

PyUnit, Sqlmap, Nmap, Parasoft Jtest, Nunit, VeraUnit, CppUnit. Bugzilla
Fiddler, JSUnit.net, OpenGrok, Wireshark, HP Fortify, CSUnit.

5) Features of White box Testing:

1. **Code coverage analysis:** White box testing helps to analyze the code coverage of an application, which helps to identify the areas of the code that are not being tested.
2. **Access to the source code:** White box testing requires access to the application's source code, which makes it possible to test individual functions, methods, and modules.
3. **Knowledge of programming languages:** Testers performing white box testing must have knowledge of programming languages like Java, C++, Python, and PHP to understand the code structure and write tests.
4. **Identifying logical errors:** White box testing helps to identify logical errors in the code, such as infinite loops or incorrect conditional statements.
5. **Integration testing:** White box testing is useful for integration testing, as it allows testers to verify that the different components of an application are working together as expected.
6. **Unit testing:** White box testing is also used for unit testing, which involves testing individual units of code to ensure that they are working correctly.
7. **Optimization of code:** White box testing can help to optimize the code by identifying any performance issues, redundant code, or other areas that can be improved.
8. **Security testing:** White box testing can also be used for security testing, as it allows testers to identify any vulnerabilities in the application's code.
9. **Verification of Design:** It verifies that the software's internal design is implemented in accordance with the designated design documents.
10. **Check for Accurate Code:** It verifies that the code operates in accordance with the guidelines and specifications.
11. **Identifying Coding Mistakes:** It finds and fix programming flaws in your code, including syntactic and logical errors.
12. **Path Examination:** It ensures that each possible path of code execution is explored and test various iterations of the code.
13. **Determining the Dead Code:** It finds and remove any code that isn't used when the programme is running normally (dead code).

6) Advantages of Whitebox Testing:

1. **Thorough Testing:** White box testing is thorough as the entire code and structures are tested.
2. **Code Optimization:** It results in the optimization of code removing errors and helps in removing extra lines of code.
3. **Early Detection of Defects:** It can start at an earlier stage as it doesn't require any interface as in the case of black box testing.
4. **Integration with SDLC:** White box testing can be easily started in Software Development Life Cycle.
5. **Detection of Complex Defects:** Testers can identify defects that cannot be detected through other testing techniques.
6. **Comprehensive Test Cases:** Testers can create more comprehensive and effective test cases that cover all code paths.
7. Testers can ensure that the code meets coding standards and is optimized for performance.

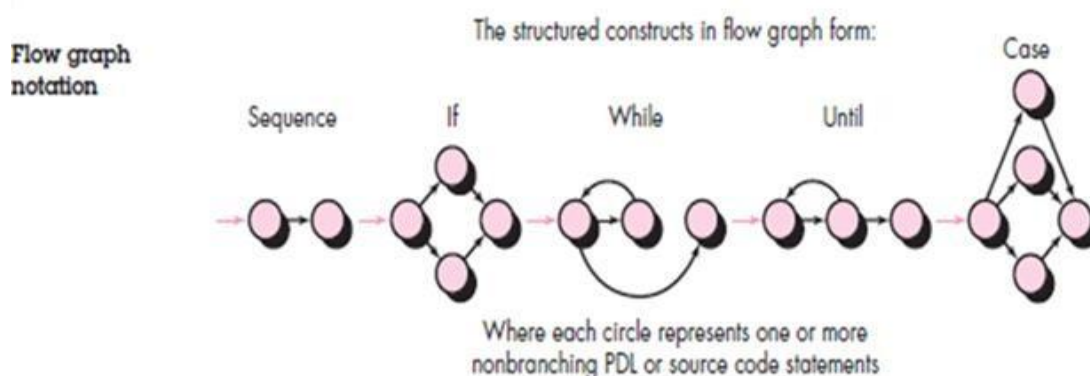
7) Disadvantages of White box Testing:

- 1.Programming Knowledge and Source Code Access:** Testers need to have programming knowledge and access to the source code to perform tests.
- 2.Overemphasis on Internal Workings:** Testers may focus too much on the internal workings of the software and may miss external issues.
- 3.Bias in Testing:** Testers may have a biased view of the software since they are familiar with its internal workings.
- 4.Test Case Overhead:** Redesigning code and rewriting code needs test cases to be written again.
- 5.Dependency on Tester Expertise:** Testers are required to have in-depth knowledge of the code and programming language as opposed to black-box testing.
- 6.Inability to Detect Missing Functionalities:** Missing functionalities cannot be detected as the code that exists is tested.
- 7.Increased Production Errors:** High chances of errors in production.

BASIS PATH TESTING

Basis path testing is a white-box testing technique first proposed by Tom McCabe. Test cases derived to exercise the basis set are guaranteed to execute every statement in the program at least one time during testing.

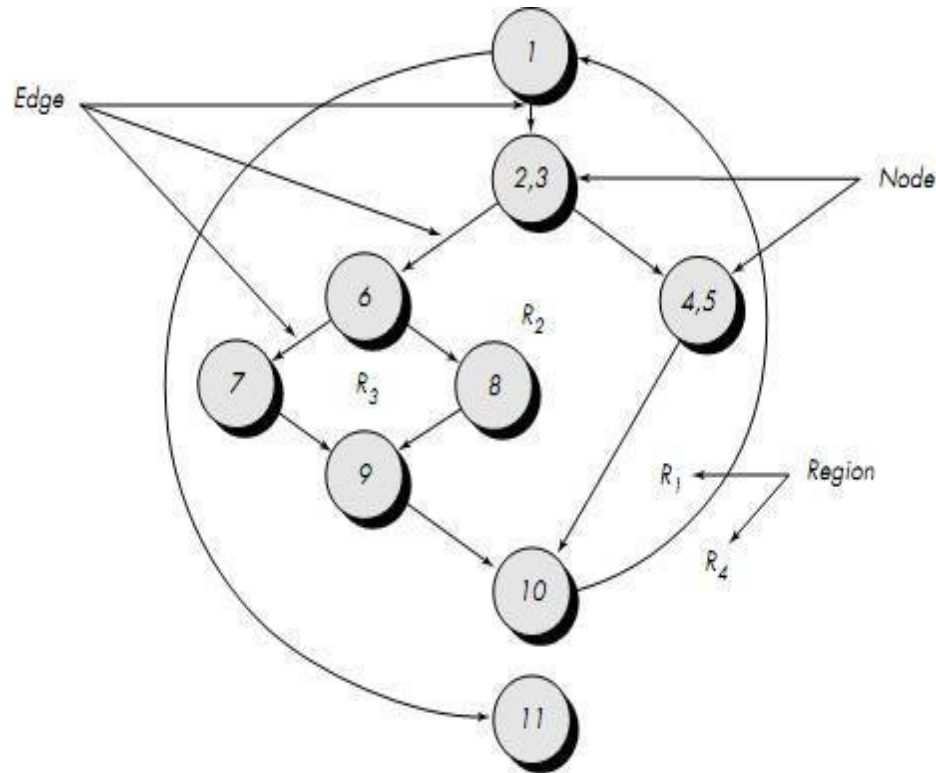
1. Flow Graph Notation: Before the basis path method can be introduced, a simple notation for the representation of control flow, called a flow graph (or program graph) must be introduced. Each circle, called a flow graph node, represents one or more procedural statements. The arrows on the flow graph, called edges or links, represent flow of control. Each node that contains a condition is called a **predicate node** and is characterized by two or more edges emanating from it.



Cyclomatic Complexity: Cyclomatic complexity is software metric that provides a quantitative measure of the logical complexity of a program.

In the context of the basis path testing method, the value computed for Cyclomatic complexity defines the number of independent paths in the basis set of a program and provides us with an upper bound for the number of tests that must be conducted to ensure that all statements have been executed at least once.

2. An independent path: is any path through the program that introduces at least one new set of processing statements or a new condition. When stated in terms of a **flow graph**, an independent path must move along at least one edge that has not been traversed before the path is defined.



In the above fig the set of independent paths are as follows:

Path 1: 1-11

Path 2: 1-2-3-4-5-10-1-11

Path 3: 1-2-3-6-8-9-10-1-11

Path 4: 1-2-3-6-7-9-10-1-11

Note that each new path introduces a new edge. The path 1-2-3-4-5-10-1-2-3-6-8-9-10-1-11

Is not considered an independent path because it is simply a combination of already specified paths and does not traverse any new edges.

Paths 1, 2, 3, and 4 constitute a basis set for the above flow graph

How is Cyclomatic complexity computed?

- The number of regions of the flow graph corresponds to the Cyclomatic complexity.
(Or)
- Cyclomatic complexity, $V(G)$, for a flow graph, G , is defined as $V(G) = E - N + 2$ Where E is the number of flow graph edges, N is the number of flow graph nodes.
(Or)
- Cyclomatic complexity, $V(G)$, for a flow graph, G , is also defined as $V(G) = P + 1$ Where P is the number of predicate nodes contained in the flow graph G .

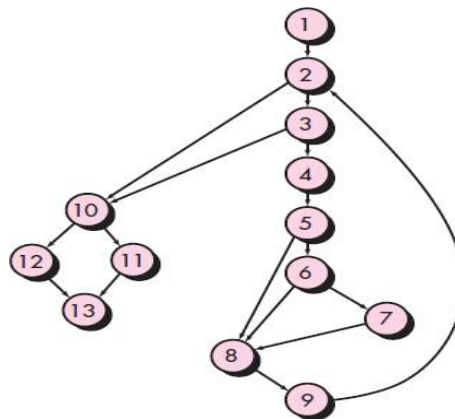
Referring once more to the above flow graph the Cyclomatic complexity can be computed using each of the algorithms just noted:

1. The flow graph has four regions.
2. $V(G) = 11 \text{ edges} - 9 \text{ nodes} + 2 = 4$.
3. $V(G) = 3 \text{ predicate nodes} + 1 = 4$. cyclomatic complexity for above flow graph.

3. Deriving Test Cases

Using the below PDL let us describe how to derive the test cases The following steps are used to derive the set of test cases

1. Using the design or code as a foundation, draw a corresponding flow graph



2. Determine the cyclomatic complexity of the resultant flow graph.

$$V(G) = 6 \text{ regions}$$

$$V(G) = 17 \text{ edges} - 13 \text{ nodes} + 2 = 6$$

$$V(G) = 5 \text{ predicate nodes} + 1 = 6$$

3. Determine a basis set of linearly independent paths.

The value of $V(G)$ provides the number of linearly independent paths through the program control structure. In the case of procedure average, we expect to specify six paths:

path 1: 1-2-10-11-13

path 2: 1-2-10-12-13

path 3: 1-2-3-10-11-13

path 4: 1-2-3-4-5-8-9-2-...

path 5: 1-2-3-4-5-6-8-9-2-...

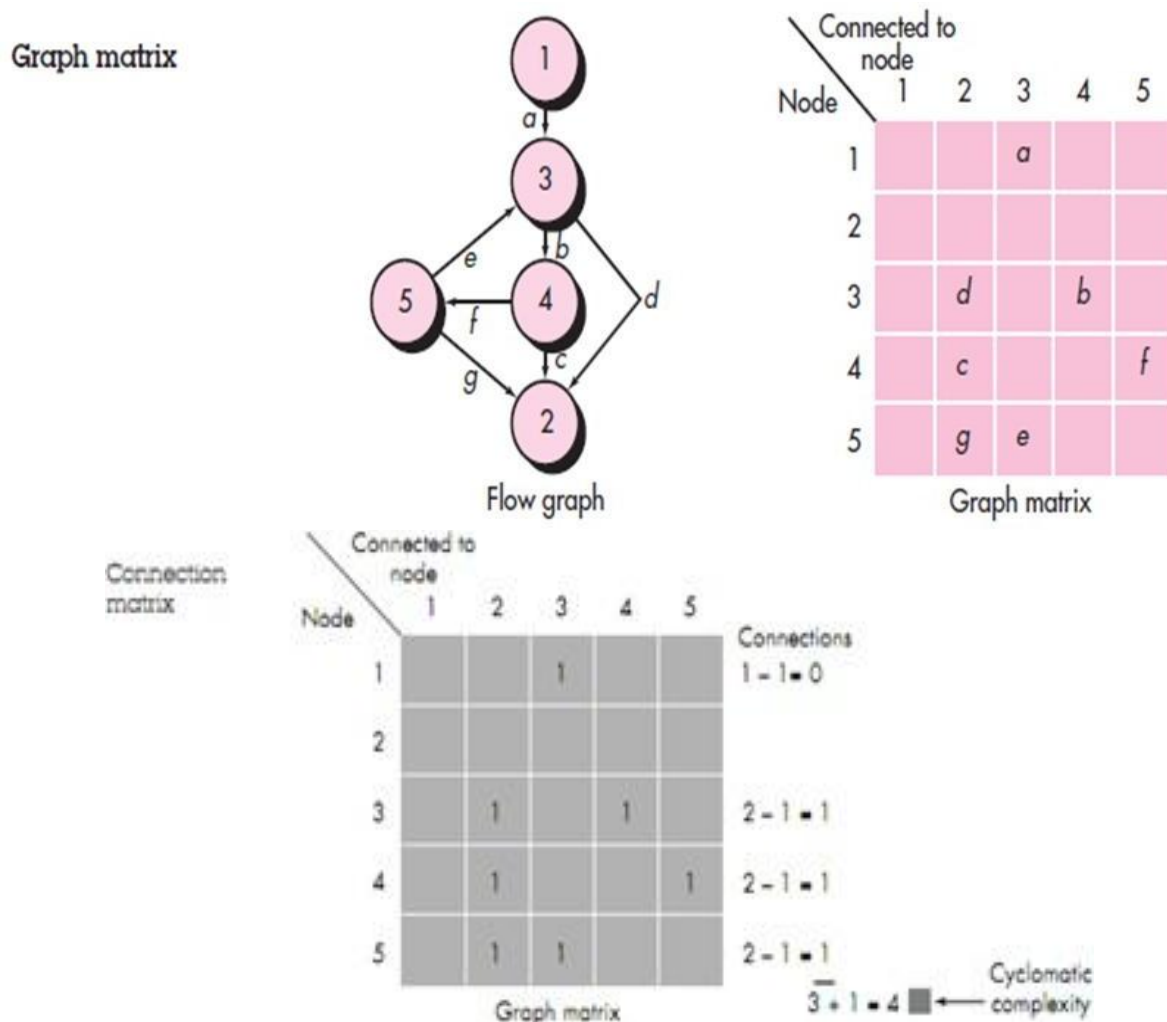
path 6: 1-2-3-4-5-6-7-8-9-2-...

The ellipsis (. . .) following paths 4, 5, and 6 indicates that any path through the remainder of the control structure is acceptable. It is often worthwhile to identify predicate nodes as an aid in the derivation of test cases. In this case, nodes 2, 3, 5, 6, and 10 are predicate nodes

4. Prepare test cases that will force execution of each path in the basis set.

Each test case is executed and compared to expected results. Once all test cases have been completed, the tester can be sure that all statements in the program have been executed at least once.

4.Graph Matrices: To develop a software tool that assists in basis path testing, a data structure, called a graph matrix, can be quite useful. A graph matrix is a square matrix whose size (i.e., number of rows and columns) is equal to number of nodes on the flow graph. Each row and column corresponds to an identified node, and matrix entries correspond to connections (an edge) between nodes. A simple example of a flow graph and its corresponding graph matrix is shown in below figure.



What is a graph matrix and how do we extend it use for testing? The graph matrix is nothing more than a tabular representation of a flow graph. By adding a link weight to each matrix entry, the graph matrix can become a powerful tool for evaluating program control structure during testing. In its simplest form, the link weight is 1 (a connection exists) or 0 (a connection does not exist).

Represented in this form, the graph matrix is called a connection matrix. Referring to above figure each row with two or more entries represents a predicate node.

Performing the arithmetic shown to the right of the connection matrix provides us with still another method for determining Cyclomatic complexity.

CONTROL STRUCTURE TESTING

It broadens testing coverage and improve quality of white-box testing.

- 1. Condition testing:** is a test case design method that exercises the logical conditions contained in a program module. A simple condition is a Boolean variable or a relational expression, possibly preceded with one NOT ($\neg$) operator. A relational expression takes the form

$$E1 <\text{relational-operator}> E2$$

where E1 and E2 are arithmetic expressions and $<\text{relational-operator}>$ is one of the following: $<$, $\leq$, $=$, $\neq$ (nonequality), $>$, or $\geq$.

A compound condition is composed of two or more simple conditions, Boolean operators, and parentheses. A condition without relational expressions is referred to as a Boolean expression.

Types of errors in a condition include the following:

- Boolean operator error (incorrect/missing/extra Boolean operators), Boolean variable error, Boolean parenthesis error, Relational operator error and Arithmetic expression error.

The purpose of condition testing is to detect not only errors in the conditions of a program but also other errors in the program

- 2. Data Flow Testing:** The data flow testing method selects test paths of a program according to the locations of definitions and uses of variables in the program.

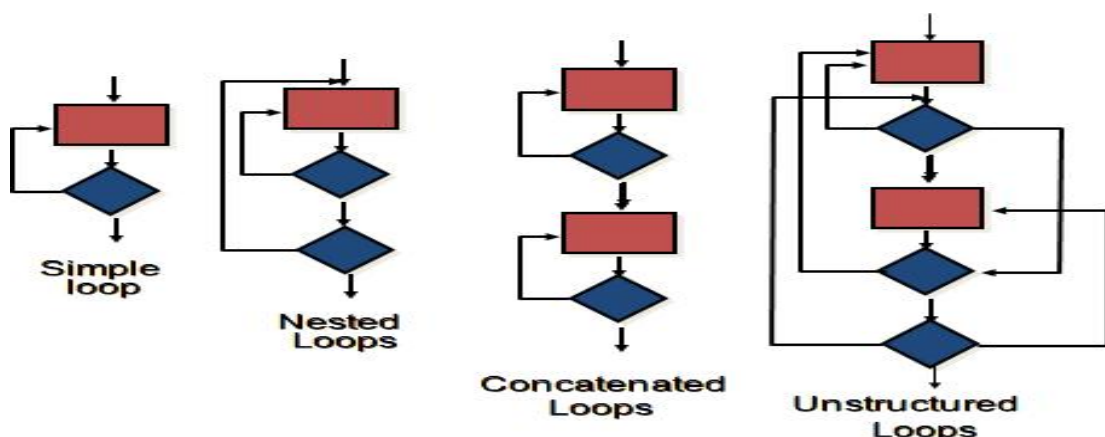
To illustrate the data flow testing approach, assume that each statement in a program is assigned a unique statement number and that each function does not modify its parameters or global variables. For a statement with S as its statement number,

$$\text{DEF}(S) = \{X \mid \text{statement } S \text{ contains a definition of } X\} \quad \text{USE}(S) = \{X \mid \text{statement } S \text{ contains a use of } X\}$$

If statement S is if or loop statement, its DEF set is empty and its USE set is based on the condition of statement S. The definition of variable X at statement S is said to be live at statement S' if there exists a path from statement S to statement S' that contains no other definition of X.

A **definition-use (DU) chain** of variable X is of the form $[X, S, S']$, where S and S' are statement numbers, X is in $\text{DEF}(S)$ and $\text{USE}(S')$, and the definition of X in statement S is live at statement S'. One simple data flow testing strategy is to require that every DU chain be covered at least once. We refer to this strategy as the **DU testing strategy**.

- 3. Loop Testing:** It is a white-box testing technique that focuses exclusively on the validity of loop constructs. Four different classes of loops can be defined: simple loops, concatenated loops, nested loops, and unstructured loops



1. **Simple loops:** The following set of tests can be applied to simple loops, where n is the maximum number of allowable passes through the loop.
 1. Skip the loop entirely.
 2. Only one pass through the loop.
 3. Two passes through the loop.
 4. m passes through the loop where $m < n$.
 5. $n - 1, n, n + 1$ passes through the loop.
2. **Nested loops:** If we were to extend the test approach for simple loops to nested loops, the number of possible tests would grow geometrically as the level of nesting increases. This would result in an impractical number of tests. Beizer suggests an approach that will help to reduce the number of tests:
 1. Start at the innermost loop. Set all other loops to minimum values.
 2. Conduct simple loop tests for the innermost loop while holding the outer loops at their minimum iteration parameter (e.g., loop counter) values.
 3. Work outward, conducting tests for the next loop, but keeping all other outer loops at minimum values and other nested loops to "typical" values.
 4. Continue until all loops have been tested.
3. **Concatenated loops:** Concatenated loops can be tested using the approach defined for simple loops, if each of the loops is independent of the other. However, if two loops are concatenated and the loop counter for loop 1 is used as the initial value for loop 2, then the loops are not independent. When the loops are not independent, the approach applied to nested loops is recommended.
4. **Unstructured loops:** Whenever possible, this class of loops should be redesigned to reflect the use of the structured programming constructs.

OBJECT-ORIENTED TESTING METHODS

The architecture of object-oriented software results in a series of layered subsystems that encapsulate collaborating classes. Each of these system elements (subsystems and classes) performs functions that help to achieve system requirements. It is necessary to test an OO system at a variety of different levels in an effort to uncover errors that may occur as classes collaborate with one another and subsystems communicate across architectural layers.

Test-case design methods for object-oriented software continue to evolve. However, an overall approach to OO test-case design has been suggested by Berard :

1. Each test case should be uniquely identified and explicitly associated with the class to be tested.
2. The purpose of the test should be stated.
3. A list of testing steps should be developed for each test and should contain:
 - a. A list of specified states for the class that is to be tested
 - b. A list of messages and operations that will be exercised as a consequence of the test

- c. A list of exceptions that may occur as the class is tested
- d. A list of external conditions (i.e., changes in the environment external to the software that must exist in order to properly conduct the test)
- e. Supplementary information that will aid in understanding or implementing the test

1.The Test Case Design Implications of OO Concepts:

The OO class is the target for test case design. Because attributes and operations are encapsulated, testing operations outside of the class is generally unproductive. Although encapsulation is an essential design concept for OO, it can create a minor obstacle when testing. As Binder notes, "Testing requires reporting on the concrete and abstract state of an object." Yet, encapsulation can make this information somewhat difficult to obtain. Unless built-in operations are provided to report the values for class attributes, a snapshot of the state of an object may be difficult to acquire.

Inheritance also leads to additional challenges for the test case designer. We have already noted that each new context of usage requires retesting, even though reuse has been achieved. In addition, multiple inheritance complicates testing further by increasing the number of contexts for which testing is required. If subclasses instantiated from a superclass are used within the same problem domain, it is likely that the set of test cases derived for the superclass can be used when testing the subclass. However, if the subclass is used in an entirely different context, the superclass test cases will have little applicability and a new set of tests must be designed.

2.Applicability of Conventional Test Case Design Methods :

The white-box testing methods can be applied to the operations defined for a class. Basis path, loop testing, or data flow techniques can help to ensure that every statement in an operation has been tested. However, the concise structure of many class operations causes some to argue that the effort applied to white-box testing might be better redirected to tests at a class level.

Black-box testing methods are as appropriate for OO systems as they are for systems developed using conventional software engineering methods. Use-cases can provide useful input in the design of black-box and state-based tests .

3.Fault-Based Testing:

The object of fault-based testing within an OO system is to design tests that have a high likelihood of uncovering plausible faults. Because the product or system must conform to customer requirements, the preliminary planning required to perform faultbased testing begins with the analysis model. The tester looks for plausible faults (i.e., aspects of the implementation of the system that may result in defects). To determine whether these faults exist, test cases are designed to exercise the design or code.

Of course, the effectiveness of these techniques depends on how testers perceive a "plausible fault." If real faults in an OO system are perceived to be "implausible," then this approach is really no better than any random testing technique. However, if the analysis and design models can provide insight into what is likely to go wrong, then fault-based testing can find significant numbers of errors with relatively low expenditures of effort.

Integration testing looks for plausible faults in operation calls or message connections. Three types of faults are encountered in this context: unexpected result, wrong operation/message used, incorrect invocation. To determine plausible faults as functions (operations) are invoked, the behavior of the operation must be examined.

Integration testing applies to attributes as well as to operations. The "behaviors" of an object are defined by the values that its attributes are assigned. Testing should exercise the attributes to determine whether proper values occur for distinct types of object behavior.

It is important to note that integration testing attempts to find errors in the client object, not the server. Stated in conventional terms, the focus of integration testing is to determine whether errors exist in the calling code, not the called code. The operation call is used as a clue, a way to find test requirements that exercise the calling code.

4. Test Cases and the Class Hierarchy:

Inheritance does not obviate the need for thorough testing of all derived classes. In fact, it can actually complicate the testing process.

Consider the following situation. A class base contains operations inherited and redefined. A class derived redefines redefined to serve in a local context. There is little doubt the derived::redefined() has to be tested because it represents a new design and new code. But does derived::inherited() have to be retested?

If derived::inherited() calls redefined and the behavior of redefined has changed, derived::inherited() may mishandle the new behavior. Therefore, it needs new tests even though the design and code have not changed. It is important to note, however, that only a subset of all tests for derived::inherited() may have to be conducted. If part of the design and code for inherited does not depend on redefined (i.e., that does not call it nor call any code that indirectly calls it), that code need not be retested in the derived class.

Base::redefined() and derived::redefined() are two different operations with different specifications and implementations. Each would have a set of test requirements derived from the specification and implementation. Those test requirements probe for plausible faults: integration faults, condition faults, boundary faults, and so forth. But the operations are likely to be similar. Their sets of test requirements will overlap. The better the OO design, the greater is the overlap. New tests need to be derived only for those derived::redefined() requirements that are not satisfied by the base::redefined() tests.

To summarize, the base::redefined() tests are applied to objects of class derived. Test inputs may be appropriate for both base and derived classes, but the expected results may differ in the derived class.

5. Scenario-Based Test Design:

Fault-based testing misses two main types of errors: (1) incorrect specifications and (2) interactions among subsystems. When errors associated with incorrect specification occur, the product doesn't do what the customer wants. It might do the wrong thing or it might omit important functionality. But in either circumstance, quality (conformance to requirements) suffers. Errors associated with subsystem interaction occur when the behavior of one subsystem creates circumstances (e.g., events, data flow) that cause another subsystem to fail.

Scenario-based testing concentrates on what the user does, not what the product does. This means capturing the tasks (via use-cases) that the user has to perform, then applying them and their variants as tests.

Scenarios uncover interaction errors. But to accomplish this, test cases must be more complex and more realistic than fault-based tests. Scenario-based testing tends to exercise multiple subsystems in a single test (users do not limit themselves to the use of one subsystem at a time).

As an example, consider the design of scenario-based tests for a text editor. Use cases follow:

Use-Case: Fix the Final Draft

Background: It's not unusual to print the "final" draft, read it, and discover some annoying errors that weren't obvious from the on-screen image. This use-case describes the sequence of events that occurs when this happens.

1. Print the entire document.
2. Move around in the document, changing certain pages.
3. As each page is changed, it's printed.
4. Sometimes a series of pages is printed.

This scenario describes two things: a test and specific user needs. The user needs are obvious: (1) a method for printing single pages and (2) a method for printing a range of pages. As far as testing goes, there is a need to test editing after printing (as well as the reverse). The tester hopes to discover that the printing function causes errors in the editing function; that is, that the two software functions are not properly independent.

Use-Case: Print a New Copy

Background: Someone asks the user for a fresh copy of the document. It must be printed.

1. Open the document.
2. Print it.
3. Close the document.

Again, the testing approach is relatively obvious. Except that this document didn't appear out of nowhere. It was created in an earlier task. Does that task affect this one?

In many modern editors, documents remember how they were last printed. By default, they print the same way next time. After the Fix the Final Draft scenario, just selecting "Print" in the menu and clicking the "Print" button in the dialog box will cause the last corrected page to print again. So, according to the editor, the correct scenario should look like this:

Use-Case: Print a New Copy

1. Open the document.
2. Select "Print" in the menu.
3. Check if you're printing a page range; if so, click to print the entire document.
4. Click on the Print button.
5. Close the document.

But this scenario indicates a potential specification error. The editor does not do what the user reasonably expects it to do. Customers will often overlook the check noted in step 3 above. They will then be annoyed when they trot off to the printer and find one page when they wanted 100. Annoyed customers signal specification bugs.

A test case designer might miss this dependency in test design, but it is likely that the problem would surface during testing. The tester would then have to contend with the probable response, "That's the way it's supposed to work!"

6. Testing Surface Structure and Deep Structure:

Surface structure refers to the externally observable structure of an OO program. That is, the structure that is immediately obvious to an end-user. Rather than performing functions, the users of many OO systems may be given objects to manipulate in some way. But whatever the interface, tests are still based on user tasks. Capturing these tasks involves understanding, watching, and talking with representative users (and as many non representative users as are worth considering).

There will surely be some difference in detail. For example, in a conventional system with a command-oriented interface, the user might use the list of all commands as a testing checklist. If no test scenarios existed to exercise a command, testing has likely overlooked some user tasks (or the interface has useless commands). In a object based interface, the tester might use the list of all objects as a testing checklist.

The best tests are derived when the designer looks at the system in a new or unconventional way. For example, if the system or product has a command-based interface, more thorough tests will be derived if the test case designer pretends that operations are independent of objects. Ask questions like, “Might the user want to use this operation—which applies only to the Scanner object—while working with the printer?” Whatever the interface style, test case design that exercises the surface structure should use both objects and operations as clues leading to overlooked tasks.

Deep structure refers to the internal technical details of an OO program. That is, the structure that is understood by examining the design and/or code. Deep structure testing is designed to exercise dependencies, behaviors, and communication mechanisms that have been established as part of the system and object design of OO software.

The analysis and design models are used as the basis for deep structure testing. For example, the object-relationship diagram or the subsystem collaboration diagram depicts collaborations between objects and subsystems that may not be externally visible. The test case design then asks: “Have we captured (as a test) some task that exercises the collaboration noted on the object-relationship diagram or the subsystem collaboration diagram? If not, why not?”

Design representations of class hierarchy provide insight into inheritance structure. Inheritance structure is used in fault-based testing.

DEBUGGING TECHNIQUES

Debugging is the process of identifying and removing defects you find in your program or product. While it’s certainly a fundamental part of QA testing, QA is a more comprehensive phase that involves ensuring other aspects of quality, such as performance and usability, not just finding bugs.

1. Ask Yourself the Right Questions:

Asking the Right Questions to Find the Root Cause of Software Bugs. Start the debugging process by defining the problem and asking yourself questions about it. For example:

What do you need the program to do?

What does it actually do?

What are the issues you’ve found?

Have you encountered these types of problems before?

What did you do to fix them?

Where and why do you think the bugs occurred?

Often, asking these questions will lead to your forming some kind of hypothesis about the nature of the errors, which will allow you to find the root cause and resolve them.

2. Pay Attention to Error Messages:

Error messages aren't just there as an annoyance — they can actually tell you exactly what the problem is with your software. So, when an error message pops up, make sure you read it, because it can give you a lot of insight into what's going on with the product.

If you're not sure what the error message means, try searching for it online. Chances are someone else has encountered the same problem in the past and could know precisely how to fix it.

3. Leverage a Debugger:

Leverage a Debugger to Identify and Resolve Software Bugs. A debugger, also known as a debugging tool or debugging mode, can be used to easily identify and correct bugs. To effectively leverage the tool, you'll need to run your program within the debugger, which allows you to monitor it in real-time and see the error when it occurs. You can pause the program while it's running to pinpoint and investigate any issues that are occurring and review your code line by line.

You'll typically use a debugger after you encounter an exception. With the tool, you can identify the problem that's taking place.

4. Log Everything:

Make sure you're logging every issue you encounter, as well as steps you take to address them and ensure your program is running correctly. Once you've documented the error, you can start mapping out potential scenarios and solutions. You should keep track of all possible steps to take and the information you need to make a decision regarding your errors. This will also allow you to navigate different potential solutions.

5. Localize the Problem:

The method of problem localization entails removing pieces of code line by line until you find the issue that is interfering with your program. While this is a somewhat painstaking and involved way of identifying the error that's taking place, it can be highly effective in determining what, exactly, is going wrong with your product. Of course, you'll need to keep repeating the process until you've tracked down the bugs.

6. Try to Replicate the Problem:

By replicating the problem, you'll find out what the nature of the problem is precisely. In fact, this can lead to you creating better, cleaner code in general since you're exercising the critical thinking skills required to find the cause of an issue.

This, of course, demands a thorough investigation of the ins and outs of the product. But once you've successfully reproduced the error that's interfering with your product's performance, usability, or functionality, fixing the problem should require far less time. In fact, most of the time, replicating the issue is the hard work, while resolving it takes only minutes.

You might turn to these solutions either because a client comes to you with a problem, and in order to fix it, you'll need to reproduce it, or as a debugging technique to have in your arsenal when you're developing products from the beginning.

7. Turn to the Community:

It's highly likely that any error you encounter is one others have encountered before you. It can be very helpful to turn to a community associated with the language, framework, or another development tool you're using to find a solution for addressing the bug you've encountered. Many development tools, such as languages like Python and frameworks like Ruby on Rails, have huge, thriving communities, offering an abundance of support to developers within them.

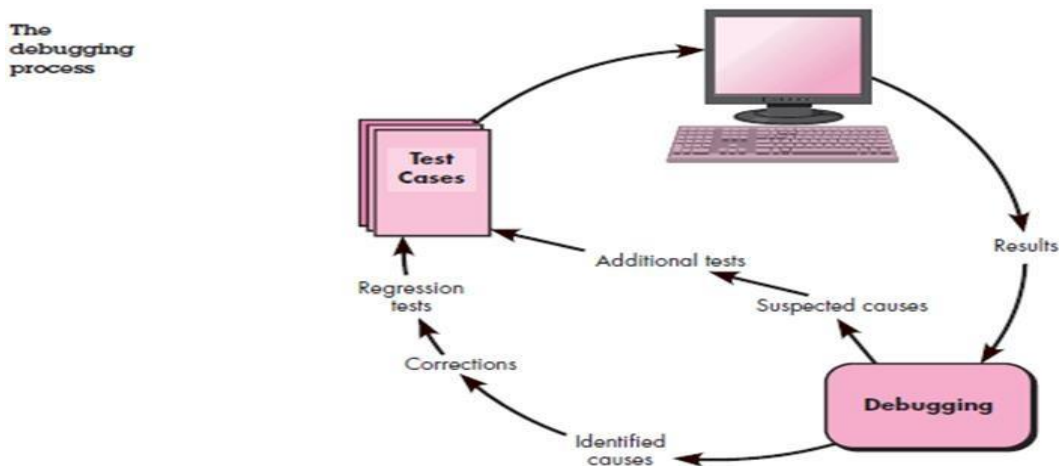
8. Test, Test, and Test Again:

The best way to spot bugs — and successfully resolve them before they derail your app — is by repeatedly testing the product. While the QA team will more thoroughly vet the product, developers themselves can script simple tests during the development phase, such as unit testing, which involves individually testing different pieces of the code — units.

These are some of the many simple debugging techniques you should have in your toolkit for building and refining software and ultimately making better products. Different methods will make more sense depending on the precise problems and scenarios you encounter, but it's a good idea to become well-versed in all of them to have more tools at your disposal and improve your development skills.

THE ART OF DEBUGGING

Debugging occurs as a consequence of successful testing. That is, when a test case uncovers an error, debugging is the process that results in the removal of the error.



Although debugging can and should be an orderly process, it is still very much an art. A software engineer, evaluating the results of a test, is often confronted with a "symptomatic" indication of a software problem. That is, the external manifestation of the error and the internal cause of the error may have no obvious relationship to one another. The poorly understood mental process that connects a symptom to a cause is debugging.

1. The Debugging Process:

The debugging process attempts to match symptom with cause, thereby leading to error correction.

The debugging process will always have one of two outcomes: (1) the cause will be found and corrected, or (2) the cause will not be found. In the latter case, the person performing debugging may suspect a cause, design a test case to help validate that suspicion, and work toward error correction in an iterative fashion.

In all likelihood, human psychology has more to do with an answer than software technology. However, a few characteristics of bugs provide some clues:

1. The symptom and the cause may be geographically remote. That is, the symptom may appear in one part of a program, while the cause may actually be located at a site that is far removed. Highly coupled program structures exacerbate this situation.
2. The symptom may disappear (temporarily) when another error is corrected.
3. The symptom may actually be caused by nonerrors (e.g., round-off inaccuracies).
4. The symptom may be caused by human error that is not easily traced.
5. The symptom may be a result of timing problems, rather than processing problems.
6. It may be difficult to accurately reproduce input conditions (e.g., a real-time application in which input ordering is indeterminate).
7. The symptom may be intermittent. This is particularly common in embedded systems that couple hardware and software inextricably.
8. The symptom may be due to causes that are distributed across a number of tasks running on different processors.

During debugging, we encounter errors that range from mildly annoying (e.g., an incorrect output format) to catastrophic (e.g. the system fails, causing serious economic or physical damage). As the consequences of an error increase, the amount of pressure to find the cause also increases. Often, pressure sometimes forces a software developer to fix one error and at the same time introduce two more.

2. Psychological Considerations:

Unfortunately, there appears to be some evidence that debugging prowess is an innate human trait. Some people are good at it and others aren't. Although experimental evidence on debugging is open to many interpretations, large variances in debugging ability have been reported for programmers with the same education and experience. Commenting on the human aspects of debugging, Shneiderman states:

Debugging is one of the more frustrating parts of programming. It has elements of problem solving or brain teasers, coupled with the annoying recognition that you have made a mistake. Heightened anxiety and the unwillingness to accept the possibility of errors increases the task difficulty. Fortunately, there is a great sigh of relief and a lessening of tension when the bug is ultimately . . . corrected.

Although it may be difficult to "learn" debugging, a number of approaches to the problem can be proposed. We examine these in the next section.

3. Debugging Approaches:

Regardless of the approach that is taken, debugging has one overriding objective: to find and correct the cause of a software error. The objective is realized by a combination of systematic evaluation, intuition, and luck. Bradley describes the debugging approach in this way:

Debugging is a straightforward application of the scientific method that has been developed over 2,500 years. The basis of debugging is to locate the problem's source [the cause] by binary partitioning, through working hypotheses that predict new values to be examined.

Take a simple non-software example: A lamp in my house does not work. If nothing in the house works, the cause must be in the main circuit breaker or outside; I look around to see whether the neighborhood is blacked out. I plug the suspect lamp into a working socket and a working appliance into the suspect circuit. So goes the alternation of hypothesis and test.

In general, three categories for debugging approaches may be proposed : (1) brute force, (2) backtracking, and (3) cause elimination.

The brute force category of debugging is probably the most common and least efficient method for isolating the cause of a software error. We apply brute force debugging methods when all else fails. Using a "let the computer find the error" philosophy, memory dumps are taken, run-time traces are invoked, and the program is loaded with WRITE statements. We hope that somewhere in the morass of information that is produced we will find a clue that can lead us to the cause of an error. Although the mass of information produced may ultimately lead to success, it more frequently leads to wasted effort and time. Thought must be expended first!

Backtracking is a fairly common debugging approach that can be used successfully in small programs. Beginning at the site where a symptom has been uncovered, the source code is traced backward (manually) until the site of the cause is found. Unfortunately, as the number of source lines increases, the number of potential backward paths may become unmanageably large.

The third approach to debugging—cause elimination—is manifested by induction or deduction and introduces the concept of binary partitioning. Data related to the error occurrence are organized to isolate potential causes. A "cause hypothesis" is devised and the aforementioned data are used to prove or disprove the hypothesis. Alternatively, a list of all possible causes is developed and tests are conducted to eliminate each. If initial tests indicate that a particular cause hypothesis shows promise, data are refined in an attempt to isolate the bug.

Each of these debugging approaches can be supplemented with debugging tools. We can apply a wide variety of debugging compilers, dynamic debugging aids ("tracers"), automatic test case generators, memory dumps, and cross-reference maps. However, tools are not a substitute for careful evaluation based on a complete software design document and clear source code.

Any discussion of debugging approaches and tools is incomplete without mention of a powerful ally—other people! Each of us can recall puzzling for hours or days over a persistent bug. A colleague wanders by and in desperation we explain the problem and throw open the listing. Instantaneously (it seems), the cause of the error is uncovered. Smiling smugly, our colleague wanders off. A fresh viewpoint, unclouded by hours of frustration, can do wonders. A final maxim for debugging might be: "When all else fails, get help!"

Once a bug has been found, it must be corrected. But, as we have already noted, the correction of a bug can introduce other errors and therefore do more harm than good. Van Vleck suggests three simple questions that every software engineer should ask before making the "correction" that removes the cause of a bug:

1. Is the cause of the bug reproduced in another part of the program? In many situations, a program defect is caused by an erroneous pattern of logic that may be reproduced elsewhere. Explicit consideration of the logical pattern may result in the discovery of other errors.
2. What "next bug" might be introduced by the fix I'm about to make? Before the correction is made, the source code (or, better, the design) should be evaluated to assess coupling of logic and data structures. If the correction is to be made in a highly coupled section of the program, special care must be taken when any change is made.
3. What could we have done to prevent this bug in the first place? This question is the first step toward establishing a statistical software quality assurance approach. If we correct the process as well as the product, the bug will be removed from the current program and may be eliminated from all future programs.

A FRAMEWORK FOR PRODUCT METRICS

1) Measures, indicators and metrics:

A **measure** is a classification unit of raw data, like a value or number like length, volume, and area. Measures are typically used to quantify some size, quantity, or intensity. Measures help to assess individual elements within a more extensive system, such as employee productivity over a certain period.

A **metric** is a measure you can quantify to monitor the progress and status of particular processes. Metrics determine the progress towards a certain goal by tracking specific data points over time.

Indicators are statistics used to measure current conditions as well as to forecast financial or economic trends.

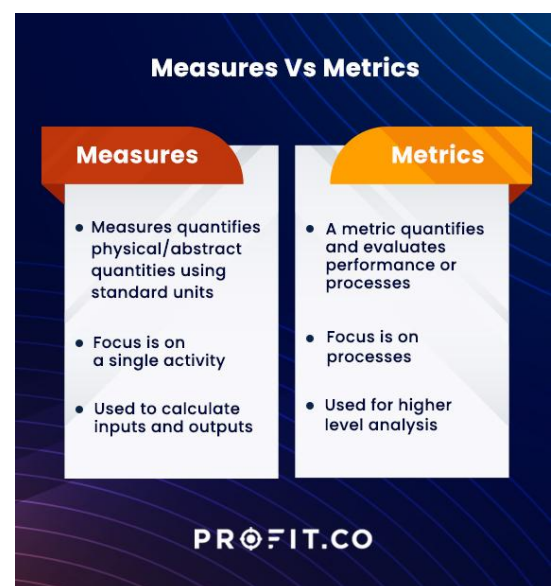
2) The Challenge of Product Metrics:

- Choosing a tool that gathers quality data and presents it in an intuitive format
- Matching the right data to your overall product goals
- Deciding the right questions to ask and identifying the right data to track to find a meaningful answer
- Interpreting the data to reveal insights and next steps to take

3) Measurement Principles:

measurement process that can be characterized by five activities:

- **Formulation.** The derivation of software measures and metrics appropriate for the representation of the software that is being considered.



- **Collection.** The mechanism used to accumulate data required to derive the formulated metrics.
- **Analysis.** The computation of metrics and the application of mathematical tools.
- **Interpretation.** The evaluation of metrics resulting in insight into the quality of the representation.

• **Feedback.** Recommendations derived from the interpretation of product metrics transmitted to the software team.

4) Goal-Oriented Software Measurement:

The Goal/Question/Metric (GQM) paradigm has been developed by Basili and Weiss [Bas84] as a technique for identifying meaningful metrics for any part of the software process. GQM emphasizes the need to

- (1) establish an explicit measurement goal that is specific to the process activity or product characteristic that is to be assessed,
- (2) define a set of questions that must be answered in order to achieve the goal, and
- (3) identify well-formulated metrics that help to answer these questions.

5) The Attributes of Effective Software Metrics:

- ✓ Simple and computable.
- ✓ Empirically and intuitively persuasive
- ✓ Consistent and objective.
- ✓ Consistent in its use of units and dimensions.
- ✓ Programming language independent.
- ✓ An effective mechanism for high-quality feedback.

METRICS FOR ANALYSIS MODEL

These metrics examine the analysis model with the intent of predicting the “size” of the resultant system. Size is an indicator of design complexity and is almost always an indicator of increased coding, integration and testing effort.

Function-Based Metrics: The function-point metric can be used effectively as a means for measuring the functionality delivered by the system. Using historical data FP metric can be used to:

1. Estimate cost or effort required to design code and test the software.
2. Predict number of errors that will be encountered during testing
3. Forecast number of components and number of projected source lines in the implemented system. Function points are derived using an empirical relationship based on countable measures of software information domain.

Number of External inputs (EI): Each external input originates from a user or is transmitted from another application. Inputs are often used to update Internal Logic Files.

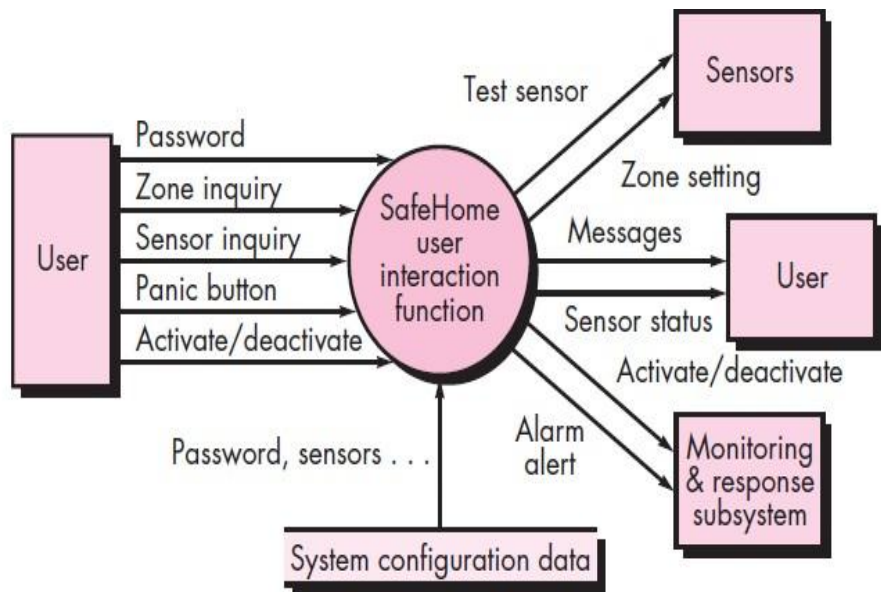
Number of External Outputs: Each external output is derived data within the application that provides information to the user. External outputs refer to reports, screens, error messages

Number of external inquiries: An external inquiry is defined as an online input that results in generation of some intermediate software response in form of an online output

Number of internal logical files: Each internal logical file is a logical grouping of data that resides within the applications boundary and is maintained via external inputs.

Number of external interface files: Each external interface file is a logical grouping of data that resides external to application but provides information that may be of use to application

A data flow model for SafeHome software



Three user inputs—password, panic button, and activate/deactivate—are shown in the figure along with two inquiries—zone inquiry and sensor inquiry. One file (system configuration file) is shown. Two user outputs (messages and sensor status) and four external interfaces (test sensor, zone setting, activate/deactivate, and alarm alert) are also present. These data, along with the appropriate complexity, are shown in below figure.

Computing function points	Information Domain Value	Count	Weighting factor			
			Simple	Average	Complex	
External Inputs (EIs)		3	3	4	6	= 9
External Outputs (EOs)		2	4	5	7	= 8
External Inquiries (EQs)		2	3	4	6	= 6
Internal Logical Files (ILFs)		1	7	10	15	= 7
External Interface Files (EIFs)		4	5	7	10	= 20
Count total						50

The count total shown in the figure must be adjusted using Equation

$$FP = \text{count total} * [0.65 + 0.01 * \sum(F_i)]$$

where count total is the sum of all FP entries obtained from Figure 19.3 and F_i ($i = 1$ to 14) are "complexity adjustment values." For the purposes of this example, we assume that $(\sum F_i)$ is 46 (a moderately complex product). Therefore,

$$FP = 50 * [0.65 + 0.01 * 46] = 56$$

METRICS FOR SPECIFICATION OF QUALITY

List of characteristics that can be used to assess the quality of the analysis model and the corresponding requirements specification: specificity (lack of ambiguity), completeness, correctness, understandability, verifiability, internal and external consistency, achievability, concision, traceability, modifiability, precision, and reusability.

We assume that there are n_r requirements in a specification, such that

$$n_r = n_f + n_{nf}$$

Where n_f is the number of functional requirements and n_{nf} is the number of non-functional (e.g., performance) requirements.

To determine the **specificity** (lack of ambiguity) of requirements

$$Q_1 = n_{ui} / n_r$$

n_{ui} is the number of requirements for which all reviewers had identical interpretations. The closer the value of Q to 1, the lower is the ambiguity of the specification

The **completeness** of functional requirements can be determined by computing the ratio

$$Q_2 = n_u / [n_i \times n_s]$$

n_u the number of unique function requirements, n_i is the number of input (stimuli) defined or implied by the specification, and n_s is the number of states specified. The Q_2 ratio measures the percentage of necessary functions that have been specified for a system

METRICS FOR DESIGN MODEL

1. **Architectural Design Metrics:** Architectural design metrics focus on characteristics of the program architecture. These metrics are black box in the sense that they do not require any knowledge of the inner workings of a particular software component.

Card and Glass define *three* software **design complexity measures**: Structural complexity, Data complexity, and System complexity.

Structural complexity of a module i is defined in the following manner:

$$S(i) = f \cdot \text{fout}(i)$$

where $\text{fout}(i)$ is the fan-out of module i . (Fan-out means number of modules directly sub-ordinate to module i)

Data complexity provides an indication of the complexity in the internal interface for a module i and is defined as

$$D(i) = v(i) / [\text{fout}(i) + 1]$$

where $v(i)$ is the number of input and output variables that are passed to and from module i .

System complexity is defined as the sum of structural and data complexity, specified as

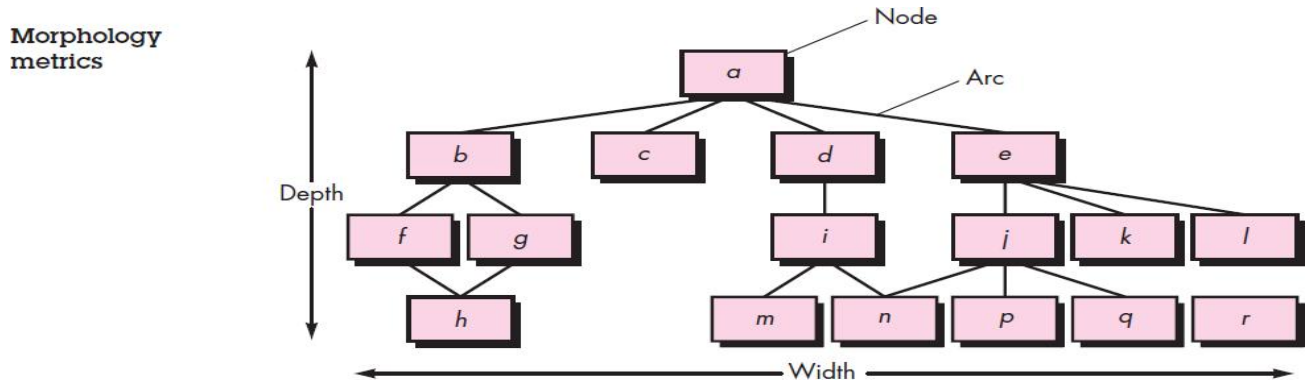
$$C(i) = S(i) + D(i)$$

As each of these complexity values increases, the overall architectural complexity of the system also increases. This leads to a greater likelihood that integration and testing effort will also increase.

Morphology (shape) metrics: It is a function of the number of modules and the number of interfaces between modules

$$\text{size} = n + a$$

where n is the number of nodes and a is the number of arcs.



For the architecture shown in above figure,

$$\text{size} = 17 + 18 = 35$$

depth = the longest path from the root (top) node to a leaf node. For the architecture shown in above figure, depth = 4.

width = maximum number of nodes at any one level of the architecture. For the architecture shown in above figure, width = 6. arc-to-node ratio, $r = a/n$, $r = 18/17 = 1.06$.

DSQI (Design Structure Quality Index): US air force has designed the DSQI. Compute s1 to s7 from data and architectural design

- S1: Total number of modules
- S2: Number of modules whose correct function depends on the data input
- S3: Number of modules whose function depends on prior processing
- S4: Number of data base items
- S5: Number of unique database items
- S6: Number of database segments
- S7: Number of modules with single entry and exit

Calculate D1 to D6 from s1 to s7 as follows:

- $D1 = 1$ if standard design is followed otherwise $D1 = 0$
- $D2(\text{module independence}) = (1 - (s2/s1))$
- $D3(\text{module not depending on prior processing}) = (1 - (s3/s1))$
- $D4(\text{Data base size}) = (1 - (s5/s4))$
- $D5(\text{Database compartmentalization}) = (1 - (s6/s4))$
- $D6(\text{Module entry/exit characteristics}) = (1 - (s7/s1))$

$$DSQI = \sum W_i D_i$$

where $i = 1$ to 6, w_i is the relative weighting of the importance of each of the intermediate values, and

$\sum w_i = 1$ (if all D_i are weighted equally, then $w_i = 0.167$).

DSQI of present design be compared with past DSQI. If DSQI is significantly lower than the average, further design work and review are indicated

2. Metrics for Object-Oriented Design: Whitmire [WHI97] describes nine distinct and measurable characteristics of an OO design:

Size: Size is defined in terms of four views: population, volume, length, and functionality

Complexity: How classes of an OO design are interrelated to one another

Coupling: The physical connections between elements of the OO design

Sufficiency: “the degree to which an abstraction possesses the features required of it, or the degree to which a design component possesses features in its abstraction, from the point of view of the current application.”

Completeness: An indirect implication about the degree to which the abstraction or design component can be reused.

Cohesion: The degree to which all operations working together to achieve a single, well-defined purpose.

Primitiveness: Applied to operations and classes, the degree to which an operation is atomic.

Similarity: The degree to which two or more classes are similar in terms of their structure, function, behavior, or purpose.

Volatility: Measures the likelihood that a change will occur.

3. Class-Oriented Metrics-The CK Metrics suite: Proposed by Chidamber and Kemerer

Weighted methods per class: Assume that n methods of complexity $c_1, c_2, \dots, c_n$ are defined for a class C . The specific complexity metric that is chosen (e.g., cyclomatic complexity) should be normalized so that nominal complexity for a method takes on a value of 1.0.

$$WMC = \sum c_i, \text{ for } i=1 \text{ to } n$$

The number of methods and their complexity are reasonable indicators of amount or effort required to implement and test a class. Although it would seem relatively straightforward to develop a count for the number of methods in a class, the problem is actually more complex than it seems. A consistent counting approach [Chu95] should be developed.

Depth of the inheritance tree: Max length from node to root of tree referring to fig DIT=4. As DIT grows low-level classes will inherit many methods, this leads to many difficulties & greater design complexity.

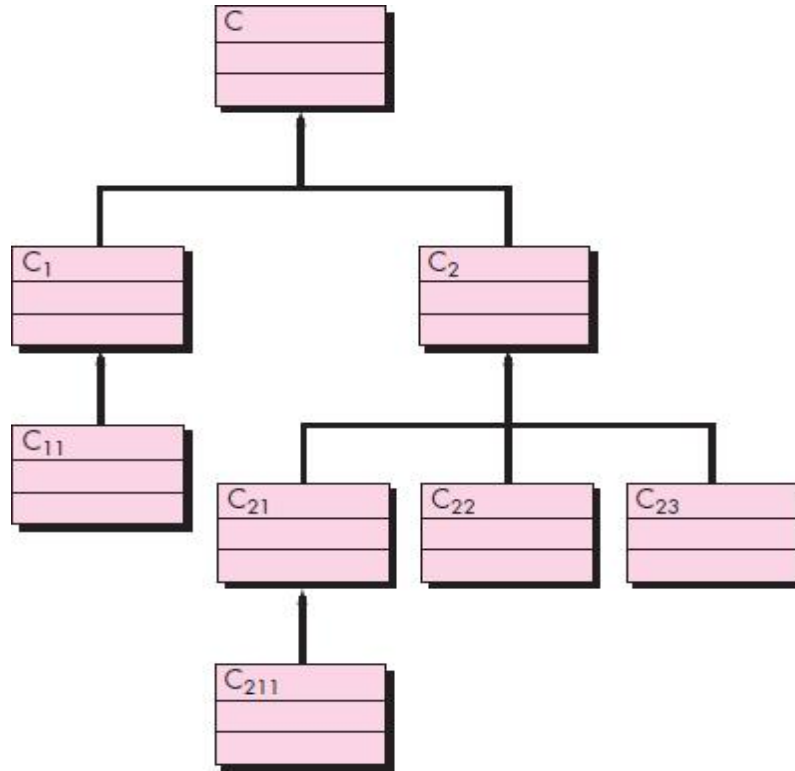


Fig: A Class Hierarchy

Number of children: The subclasses that are immediately subordinate to a class in class hierarchy are termed its children. As NOC grows reuse increases but abstraction represented by parent class is diluted if some children are not appropriate members of parent class.

Coupling between object classes: As CBO increases reusability decreases

Response for a class: A set of methods that can potentially be executed in response to a message. RFC is no. of methods in response set. As RFC increases complexity increases.

Lack of cohesion in methods: LCOM is no. of methods that access one or more of same attributes. If no methods access same attribute LCOM=0

4. Class-Oriented Metrics: The MOOD Metrics Suite

Method Inheritance Factor:

$$MIF = \sum M_i(C_i) / \sum M_d(C_i)$$

where the summation occurs over $i = 1$ to T_c . T_c is defined as the total number of classes in the architecture; C_i is a class within the architecture; and

$$M_d(C_i) = M_d(C_i) + M_i(C_i)$$

where

$M_d(C_i)$ = the number of methods that can be invoked in association with C_i .

$M_d(C_i)$ = the number of methods declared in the class C_i .

$M_i(C_i)$ = the number of methods inherited (and not overridden) in C_i .

Coupling factor:

$$CF = \sum_i \sum_j is_client(C_i, C_j) / (T_c^2 - T_c)$$

where the summations occur over $i = 1$ to T_c and $j = 1$ to T_c . The function

$is_client = 1$, if and only if a relationship exists between the client class, C_c , and the server class, C_s , and $C_c \neq C_s$
 $= 0$, otherwise

If CF increases the complexity of OO software also increases.

5. Class-Oriented Metrics Proposed by Lorenz and Kidd

Lorenz and Kidd divide class-based metrics into four broad categories

- ❖ **Size-Oriented Metrics:** focus on count of attributes and operations for an individual class
- Inheritance-Based Metrics:* focus on manner in which operations are reused through class hierarchy.

- ✓ *Metrics For Class Internal:* focus on cohesion
- ✓ *Metrics For External:* focus on coupling

6. Component-Level design metrics

- ✓ *Cohesion metrics:* a function of data objects and the focus of their definition
- ✓ *Coupling metrics:* a function of input and output parameters, global variables, and modules.
- ✓ *Complexity metrics:* hundreds have been proposed (e.g., Cyclomatic complexity)

7. Operation-Oriented Metrics

- ✓ average operation size
- ✓ operation complexity
- ✓ average number of parameters per operation

8. Interface Design Metrics

- ✓ *Layout appropriateness:* a function of layout entities, the geographic position and the “cost” of making transitions among entities. This is a worthwhile design metric for interface design.

METRICS FOR SOURCE CODE

Primitive measure that may be derived after the code is generated or estimated once design is complete.

$$\text{Length } N = n_1 \log_2 n_1 + n_2 \log_2 n_2$$

$$\text{Program volume } V = N \log_2 (n_1 + n_2)$$

$$\text{Volume ratio } L = 2/n_1 * n_2/N_2$$

Where, n_1 = the number of distinct operators that appear in a program

n_2 = the number of distinct operands that appear in a program

N_1 = the total number of operator occurrences.

N_2 = the total number of operand occurrence.

METRICS FOR TESTING

Testing effort can be estimated using metrics derived from Halstead measures . Using the definitions for program volume V and program level PL, Halstead effort e can be computed as

$$\text{Program Level and Effort (PL)} = 1/[(n_1 / 2) \times (N_2 / n_2 l)]$$
$$e = V/PL$$

Metrics for Object-Oriented Testing:

1. **Lack of cohesion in methods (LCOM):** The higher the value of LCOM, the more states must be tested to ensure that methods do not generate side effects.
2. **Percent public and protected (PAP):** Public attributes are inherited from other classes and therefore are visible to those classes. Protected attributes are accessible to methods in subclasses.
3. **Public access to data members (PAD):** This metric indicates the number of classes (or methods) that can access another class's attributes, a violation of encapsulation. High values for PAD lead to the potential for side effects among classes. Tests must be designed to ensure that such side effects are uncovered.
4. **Number of root classes (NOR):** This metric is a count of the distinct class hierarchies that are described in the design model. Test suites for each root class and the corresponding class hierarchy must be developed. As NOR increases, testing effort also increases.
5. **Fan-in (FIN):** When used in the OO context, fan-in in the inheritance hierarchy is an indication of multiple inheritance. $FIN > 1$ indicates that a class inherits its attributes and operations from more than one root class. $FIN > 1$ should be avoided when possible.

METRICS FOR MAINTENANCE

IEEE standard suggests a software maturity index that provides indication of stability of software product. The Software Maturity Index, SMI, is defined as:

$$SMI = \frac{M_T - (F_a + F_c + F_d)}{M_T}$$

Where

M_t = the number of modules in the current release

F_c = the number of modules in the current release that have been changed

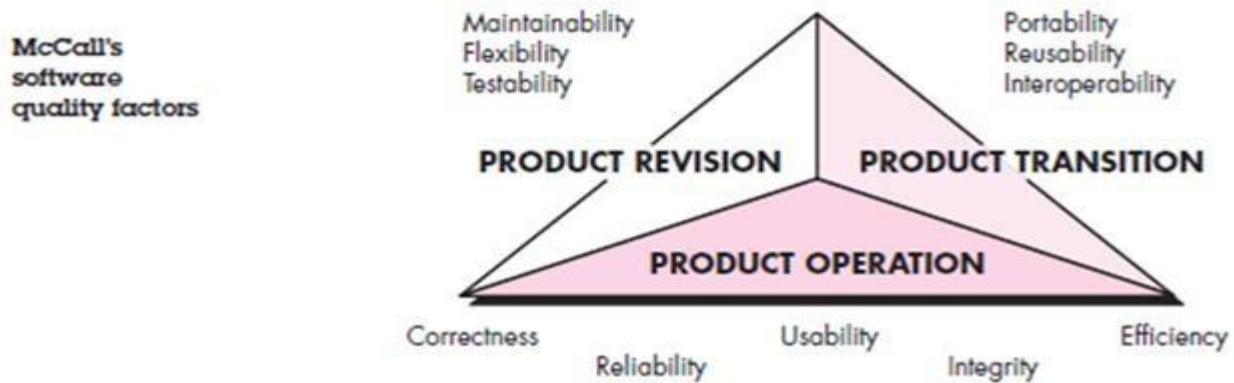
F_a = the number of modules in the current release that have been added.

F_d = the number of modules from the preceding release that were deleted in the current release

SOFTWARE QUALITY: Software quality is defined as the conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software

McCall's Quality Factors: Factors that affect software quality can be categorized in two broadgroups:

1. Factors that can be directly measured (e.g. defects uncovered during testing)
2. Factors that can be measured only indirectly (e.g. usability or maintainability)



Figure, focus on three important aspects of a software product: its operational characteristics, its ability to undergo change, and its adaptability to new environments.

McCall and his colleagues provide the following descriptions:

1. **Correctness:** The extent to which a program satisfies its specification and fulfills the customer's mission objectives.
2. **Reliability:** The extent to which a program can be expected to perform its intended function with required precision.
3. **Efficiency:** The amount of computing resources and code required by a program to perform its function.
4. **Integrity:** Extent to which access to software or data by unauthorized persons can be controlled.
5. **Usability:** Effort required to learn, operate, prepare input, and interpret output of a program.
6. **Maintainability:** Effort required to locate and fix an error in a program.
7. **Flexibility:** Effort required to modify an operational program.
8. **Testability:** Effort required to test a program to ensure that it performs its intended function.
9. **Portability:** Effort required to transfer the program from one hardware and/or software system environment to another.
10. **Reusability:** Extent to which a program [or parts of a program] can be reused in other applications related to the packaging and scope of the functions that the program performs.
11. **Interoperability:** Effort required to couple one system to another.

ISO 9126 Quality Factors

The ISO 9126 standard was developed in an attempt to identify the key quality attributes for computer software. The standard identifies six key quality attributes:

2. **Functionality:** The degree to which the software satisfies stated needs as indicated by the following sub attributes: *suitability, accuracy, interoperability, compliance, and security*.
3. **Reliability:** The amount of time that the software is available for use as indicated by the following sub attributes: *maturity, fault tolerance, recoverability*.

4. **Usability:** The degree to which the software is easy to use as indicated by the following sub attributes: *understandability, learnability, operability*.
5. **Efficiency:** The degree to which the software makes optimal use of system resources as indicated by the following sub attributes: *time behavior, resource behavior*.
6. **Maintainability:** The ease with which repair may be made to the software as indicated by the following sub attributes: *analyzability, changeability, stability, testability*.
7. **Portability:** The ease with which the software can be transposed from one environment to other as indicated by the following attributes: *adaptability, installability, conformance, replaceability*.

Measures, Metrics and Indicators

- ✓ **A *measure*** provides a quantitative indication of the extent, amount, dimension, capacity, or size of some attribute of a product or process
- ✓ **The IEEE glossary defines a *metric*** as “a quantitative measure of the degree to which a system, component, or process possesses a given attribute.”
- ✓ **An *indicator*** is a metric or combination of metrics that provide insight into the software process, a software project, or the product itself.

SOFTWARE QUALITY ASSURANCE

Elements of SQA: Software quality assurance encompasses a broad range of concerns and activities that focus on the management of software quality. These can be summarized as:

1. **Standards:** The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. Standards may be adopted voluntarily by a software engineering organization or imposed by the customer or other stakeholders. The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.
2. **Reviews and Audits:** Technical reviews are a quality control activity performed by software engineers for software engineers. Their intent is to uncover errors. Audits are a type of review performed by SQA personnel with the intent of ensuring that quality guidelines are followed for software engineering work.
3. **Testing:** Software testing is a quality control function that has one primary goal—to find errors. The job of SQA is to ensure that testing is properly planned and efficiently conducted to find the errors.
4. **Error/defect collection and analysis:** SQA collects and analyzes error/defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminate them.
5. **Change management:** Change is one of the most disruptive aspects of any software project. If it is not properly managed, change can lead to confusion, and thus poor quality. SQA ensures that adequate change management practices have been instituted.
6. **Education:** Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement and is a key proponent and sponsor of educational/training programs.
7. **Vendor management:** Three categories of software are acquired from external software vendors— *shrink-wrapped packages* (e.g., Microsoft Office), a *tailored shell* that provides a basic skeletal structure that is custom tailored to the needs of a purchaser, and *contracted software* that is custom designed and constructed from specifications provided by the customer organization. The job of SQA organization is to ensure that high-quality software results by suggesting specific quality practices that the vendor should follow, and incorporating quality mandates as part of any contract with an external vendor.
8. **Security management:** With the increase in cyber crime, every software organization should institute policies that protect data at all levels, establish firewall protection for WebApps, and ensure that software has not been tampered with internally. SQA ensures that appropriate process and technology are used to achieve software security.
9. **Safety:** As software is almost always a pivotal component of human rated systems (e.g.,

automotive or aircraft applications), the impact of hidden defects can be catastrophic. SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.

- 10. Risk management:** Although the analysis and mitigation of risk is the concern of software engineers, the SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

SQA Tasks:

The Software Engineering Institute recommends a set of SQA actions that address quality assurance planning, oversight, record keeping, analysis, and reporting. These actions are performed by an independent SQA group that:

- 1. Prepares an SQA plan for a project:** The plan is developed as part of project planning and is reviewed by all stakeholders. Quality assurance actions performed by the SE team and the SQA group are governed by the plan. The plan identifies evaluations to be performed, audits and reviews to be conducted, standards that are applicable to the project, procedures for error reporting and tracking, work products that are produced by the SQA group and feedback that will be provided to the software team.
- 2. Participates in the development of the project's software process description:** The software team selects a process for the work to be performed. The SQA group reviews the process description for compliance with organizational policy, internal software standards, externally imposed standards (e.g., ISO-9001), and other parts of the software project plan.
- 3. Reviews software engineering activities to verify compliance with the defined software process:** The SQA group identifies, documents, and tracks deviations from the process and verifies that corrections have been made.
- 4. Audits designated software work products to verify compliance with those defined as part of the software process:** The SQA group reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made; and periodically reports the results of its work to the project manager.
- 5. Ensures that deviations in software work and work products are documented and handled according to a documented procedure:** Deviations may be encountered in the project plan, process description, applicable standards, or software engineering work products.
- 6. Records any noncompliance and reports to senior management:** Noncompliance items are tracked until they are resolved. In addition to these actions, the SQA group coordinates the control and management of change and helps to collect and analyze software metrics.

Formal Approaches to SQA: Software quality is everyone's job and it can be achieved through competent software engineering practice as well as through the application of technical reviews, a multi-tiered testing strategy, better control of software

work products and the changes made to them, and the application of accepted software engineering standards. In addition, quality can be defined in terms of a broad array of quality attributes and measured using a variety of indices and metrics.

Over the past three decades, a small, but vocal, segment of the software engineering community has argued that a more formal approach to software quality assurance is required. It can be argued that a computer program is a mathematical object. A rigorous syntax and semantics can be defined for every programming language, and a rigorous approach to the specification of software requirements is available.

If the requirements model and the programming language can be represented in a rigorous manner, it should be possible to apply mathematic proof of correctness to demonstrate that a program conforms exactly to its specifications.

Statistical Software Quality Assurance: Statistical quality assurance reflects a growing trend throughout industry to become more quantitative about quality. For software, statistical quality assurance implies the following steps:

1. Information about software errors and defects is collected and categorized.
2. An attempt is made to trace each error and defect to its underlying cause (e.g., nonconformance to specifications, design error, violation of standards, poor communication with the customer).
3. Using the Pareto principle isolate the 20 percent (80 percent of the defects can be traced to 20 percent of all possible causes).
4. Once the vital few causes have been identified, move to correct the problems that have caused the errors and defects.

Software Reliability: *Software reliability* is defined in statistical terms as “the probability of failure-free operation of a computer program in a specified environment for a specified time”.

Measures of Reliability and Availability: If we consider a computer-based system, a simple measure of reliability is *meantime-between-failure* (MTBF): $MTBF = MTTF + MTTR$, where the acronyms MTTF and MTTR are *mean-time-to-failure* and *mean-time-to-repair*, respectively.

MTBF is a far more useful measure than other quality-related software metrics. An end user is concerned with failures, not with the total defect count. Because each defect contained within a program does not have the same failure rate, the total defect count provides little indication of the reliability of a system.

However, MTBF can be problematic for two reasons:

- (1) It projects a time span between failures, but does not provide us with a projected failure rate, and
- (2) MTBF can be misinterpreted to mean average life span even though this is *not* what it implies.

An alternative measure of reliability is *failures-in-time* (FIT) — a statistical measure of how many failures a component will have over one billion hours of operation. Therefore, FIT is equivalent to one failure in every billion hours of operation.

In addition to a reliability measure, you should also develop a measure of availability. *Software availability* is the probability that a program is operating according to requirements at a given point in time and is defined as

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \times 100\%$$

The MTBF reliability measure is equally sensitive to MTTF and MTTR. The availability measure is somewhat more sensitive to MTTR, an indirect measure of the maintainability of software.

The ISO 9000 Quality Standards: A *quality assurance system* may be defined as the organizational structure, responsibilities, procedures, processes, and resources for implementing quality management. Quality assurance systems are created to help organizations ensure their products and services satisfy customer expectations by meeting their specifications.

ISO 9000 describes quality assurance elements in generic terms that can be applied to any business regardless of the products or services offered. To become registered to one of the quality assurance system models contained in ISO 9000, a company's quality system and operations are scrutinized by third-party auditors for compliance to the standard and for effective operation. Upon successful registration, a company is issued a certificate from a registration body represented by the auditors.

Semiannual surveillance audits ensure continued compliance to the standard. The requirements delineated by ISO 9001:2000 address topics such as management responsibility, quality system, contract review, design control, document and data control, product identification and traceability, process control, inspection and testing, corrective and preventive action, control of quality records, internal quality audits, training, servicing, and statistical techniques.

In order for a software organization to become registered to ISO 9001:2000, it must establish policies and procedures to address each of the requirements just noted (and others) and then be able to demonstrate that these policies and procedures are being followed.

The following outline defines the basic elements of the ISO 9001:2000 standard.

- ✓ Establish the elements of a quality management system.
 - Develop, implement, and improve the system.
 - Define a policy that emphasizes the importance of the system.
- ✓ Document the quality system.
 - Describe the process.
 - Produce an operational manual.

- Develop methods for controlling (updating) documents.
- Establish methods for record keeping.
- ✓ Support quality control and assurance.
 - Promote the importance of quality among all stakeholders.
 - Focus on customer satisfaction.
 - Define a quality plan that addresses objectives, responsibilities, and authority.
 - Define communication mechanisms among stakeholders.
- ✓ Establish review mechanisms for the quality management system.
 - Identify review methods and feedback mechanisms.
 - Define follow-up procedures.
- ✓ Identify quality resources including personnel, training, and infrastructure elements.
 - Establish control mechanisms.
 - For planning
 - For customer requirements
 - For technical activities (e.g., analysis, design, testing)
 - For project monitoring and management
- ✓ Define methods for remediation.
 - Assess quality data and metrics.
 - Define approach for continuous process and quality improvement.

The ISO 9000 standard is based on seven quality management principles which help the organization to ensure that their products or services are aligned with the customer needs’.

Seven principles of ISO 9000 are depicted in the below image:

The SQA Plan: *SQA Plan* provides a road map for instituting software quality assurance. Developed by the SQA group the plan serves as a template for SQA activities that are instituted for each software project.



Software Quality Assurance (SQA) plan is an outline of quality measures to ensure quality levels within a software development effort.

The plan is used as a baseline to compare actual levels of quality during development with planned levels of quality. If levels of quality are not within the planned quality levels, management will respond appropriately as documented within the plan.

The plan provides a framework and guidelines for development of understandable and maintainable code. These ingredients help ensure quality sought in S/W project. SQA plan also provides procedures for ensuring that quality software will be produced or maintained

in-house or under contract. These procedures affect planning, designing, writing, testing, documenting, storing, maintaining computer software. It should be organized in this way as the plan ensures quality of S/W rather than describing specific procedures for developing and maintaining it.

Steps to Develop and Implement a Software Quality Assurance Plan

Step 1: Document the Plan: SQA plan should include the following sections:

1. ***Purpose Section:*** It delineates the specific purpose and scope of the particular SQA plan. It should list the names of the software items covered by the SQA plan and the intended use of the software. It states the portion of the software life cycle covered by the SQA plan for each software item specified.
2. ***Reference Document Section:*** It provides a complete list of documents referenced elsewhere in the text of the SQA plan.
3. ***Management Section:*** It describes the project's organizational structure, tasks, responsibilities.
4. ***Documentation Section:*** It identifies the documentation governing the development, verification and validation, use, and maintenance of the software. It also states how the documents are to be checked for adequacy. This includes the criteria and the identification of the review or audit by which the adequacy of each document will be confirmed.
5. ***Standards, Practices, Conventions, and Metrics Section:*** It identifies the standards, practices, conventions, and metrics to be applied, and also states how compliance with these items is to be monitored and assured.
6. ***Reviews and Inspections Section:*** It defines the technical and managerial reviews, walkthroughs, and inspections to be conducted. It also states how the reviews, walkthroughs, and inspections are to be accomplished, including follow-up activities and approvals.
7. ***Software Configuration Management Section:*** It is addressed in detail in the project's software configuration management plan.
8. ***Problem Reporting and Corrective Action Section:*** It is addressed in detail in the project's software configuration management plan.
9. ***Tools, Techniques, and Methodologies Section:*** It identifies the special software tools, techniques, and methodologies that support SQA, states their purposes, and describes their use.
10. ***Code Control Section:*** It defines the methods and facilities used to maintain, store, secure, and document the controlled versions of the identified software during all phases of development. This may be implemented in conjunction with a computer program library or may be provided as a part of the software configuration management plan.
11. ***Media Control Section:*** It describes the methods and facilities to be used to identify the

media for each computer product and the documentation required to store the media, including the copy and restores process, and protects the computer program physical media from unauthorized access or inadvertent damage or degradation during all phases of development. This may be provided by the software configuration management plan.

12. **Supplier Control Section:** It states the provisions for ensuring that S/W provided by suppliers meets established requirements. In addition, it should specify the methods that will be used to ensure that S/W supplier receives adequate and complete requirements. For previously developed software, it describes the methods to be used to ensure the suitability of the product for use with the software items covered by SQA plan. For S/W to be developed, supplier will be required to prepare and implement SQA plan in accordance with this standard. It will also state the methods to be employed to ensure that the developers comply with the requirements of this standard.
13. **Records Collection, Maintenance, and Retention Section:** It identifies the SQA documentation to be retained. It states the methods and facilities to assemble, safeguard, and maintain this documentation, and will designate the retention period. The implementation of the SQA plan involves the necessary approvals for the plan as well as development of a plan for execution. The subsequent evaluation of the SQA plan will be performed as a result of its execution.
14. **Testing Methodology:** It defines testing approach, techniques, automated tools that will be used.

Step 2: Obtain Management Acceptance: Management participation is necessary for successful implementation of SQA plan. Management is responsible for both ensuring the quality of S/W project and for providing the resources needed for S/W development.

The level of management commitment required for implementing SQA plan depends on scope of the project. If a project spans organizational boundaries, approval should be obtained from all affected departments. Once approval has been obtained, SQA plan is placed under configuration control.

In the management approval process, management relinquishes tight control over software quality to the SQA plan administrator in exchange for improved software quality. Software quality is often left to software developers. Quality is desirable, but management may express concern as to the cost of a formal SQA plan. Staff should be aware that management views the program as a means of ensuring software quality and not as an end in itself.

To address management concerns, software life-cycle costs should be formally estimated for projects implemented both with and without a formal SQA plan. In general, implementing a formal SQA plan makes economic and management sense.

Step 3: Obtain Development Acceptance: Because the software development and maintenance personnel are the primary users of SQA plan, their approval and cooperation in implementing the plan are essential. The software project team members must adhere to the project SQA plan; everyone must accept it and follow it.

No SQA plan is successfully implemented without the involvement of the software team members and their managers in the development of the plan. Because project teams generally have only a few members, all team members should actively participate in writing the SQA plan. When projects become much larger (i.e., encompassing entire divisions or departments), representatives of project subgroups should provide input. Constant feedback from representatives to team members helps gain acceptance of the plan

Step 4: Plan for Implementation of the SQA Plan: The process of planning, formulating, and drafting SQA plan requires staff and word-processing resources. The individual responsible for implementing SQA plan must have access to these resources. In addition, the commitment of resources requires management approval and, consequently, management support. To facilitate resource allocation, management should be made aware of any project risks that may impede the implementation process (e.g., limited availability of staff or equipment). A schedule for drafting, reviewing, and approving the SQA plan should be developed.

Step 5: Execute the SQA Plan: The actual process of executing SQA plan by the software development and maintenance team involves determining necessary audit points for monitoring it. The auditing function must be scheduled during the implementation phase of the software product so that improper monitoring of the software project will not hurt the SQA plan. Audit points should occur either periodically during development or at specific project milestones (e.g., at major reviews or when part of the project is delivered).

A standard for SQA plans recommends a structure that identifies:

1. The purpose and scope of the plan
2. A description of all software engineering work products (e.g., models, documents, source code) that fall within the purview of SQA
3. All applicable standards and practices that are applied during the software process
4. SQA actions and tasks (including reviews, audits) and their placement throughout the software process
5. The tools and methods that support SQA actions and tasks
6. Software configuration management procedures
7. Methods for assembling, safeguarding, and maintaining all SQA-related records, and
8. Organizational roles and responsibilities relative to product quality.